# Introduction à l’informatique (R et Python) et à LaTeX

*Compétences en ligne de commande, analyse de données avec R et Python, et documents professionnels avec LaTeX.*

## Mes cours




18 leçons

### Programmation de scripts Bash

De votre première commande dans le terminal à des scripts qui bouclent, ramifient, traitent du texte et gèrent les signaux.



17 leçons

### Programmation R

Structures de données, nettoyage, visualisation, fonctions et premier aperçu de la modélisation.



17 leçons

### Programmation Python

L’équivalent Python du cours de R : tableaux et DataFrames, nettoyage, graphiques, fonctions et premier aperçu de la modélisation.

22 leçons

## LaTeX

Composez des documents en écrivant plutôt qu’en cliquant : structure, formules, tableaux, figures et packages.



8 leçons

## Préparation à l’épidémiologie mathématique

Modèles SIR et SEIR, R0, vaccination, flambées stochastiques et ajustement de modèles, en R et en Python.

# Ressources

○ ○ ○ terminal

```
git clone https://github.com/seknewna/Introduction-to-Computing-and-LaTeX.git
cd Introduction-to-Computing-and-LaTeX
```

```
bash-scripting/scripts/*.sh
r-programming/chapters/*.Rmd
python/chapters/*.py
python/notebooks/*.ipynb
latex/index.html
mathepi/r/*.Rmd
mathepi/python/*.py
```

Chaque script et chaque chapitre ci-dessous est aussi affiché en entier sur cette page — ne clonez le dépôt que si vous souhaitez les exécuter en local.




# Programmation de scripts Bash

18 leçons · Enseignement : Dr. Lema Logamou Seknewna

18 chapitres, de votre première commande dans le terminal jusqu’aux signaux (trap) et aux tubes nommés. Contenu du cours adapté du « Complete Bash Scripting Course » gratuit de Dave Eddy (ysap.sh), code source sous licence MIT.

---

Chapitre 00 · 1 sur 18

## Introduction

Chaque script commence par une ligne shebang qui indique au système quel interpréteur utiliser. Ce chapitre fixe les attentes pour le cours et prépare votre environnement avant que vous n’écriviez la moindre vraie ligne de script.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/00-introduction.sh

```
#!/usr/bin/env bash
#
# Chapter 00 — Introduction
# Real commands referenced in this lesson of the course.

echo "This course lives entirely in the terminal — no separate app needed."
echo "Every later script in this repo starts with the same shebang line:"
echo '  #!/usr/bin/env bash'
```

# Terminal et fichiers : les bases

Avant les scripts, il faut être à l'aise dans le terminal lui-même : se déplacer dans le système de fichiers, créer et supprimer des fichiers, afficher les fichiers cachés (dotfiles), chercher dans les fichiers avec grep, et parcourir une sortie trop longue pour tenir sur un écran.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/01-terminal-and-file-basics.sh

```
#!/usr/bin/env bash
#
# Chapter 01 — Terminal & File Basics
# Real commands from this lesson: cd, mkdir, touch, cat, echo, rm, pwd,
# cp, alias, man, ls -a, ls -l, grep -A/-B/-C/-i/-o/-v, less/more.

mkdir -p foo && cd foo

pwd                          # confirm where you are (CWD)
touch lesson-1.txt
echo hi > lesson-1.txt
cat lesson-1.txt
cp lesson-1.txt lesson-2.txt
rm lesson-1.txt lesson-2.txt

echo
echo "Hidden files start with a dot and are skipped by plain ls:"
touch .lesson.txt
ls                           # .lesson.txt is not listed
ls -a                        # now it shows up
ls -l .lesson.txt
rm .lesson.txt

echo
echo "grep context flags — build a small file to search:"
printf 'a\nb\nc\n' > file.txt
grep -A1 b file.txt          # 1 line of context After the match
grep -i B file.txt           # case-Insensitive
grep -o b file.txt           # Only print the matched text
grep -v b file.txt           # Invert — show non-matching lines
rm file.txt

echo
echo "man rm shows -i (confirm before delete) — worth aliasing:"
echo "  alias rm='rm -i'"

cd .. && rm -rf foo
```

# Pages man, variables et permissions

Apprenez à lire le manuel d'une commande plutôt que de deviner, à distinguer les programmes des commandes intégrées (builtins) du shell, à définir vos premières variables et à comprendre les bits de permission lecture/écriture/exécution qui déterminent ce qu'un fichier peut faire.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/02-man-pages-variables-permissions.sh

```
#!/usr/bin/env bash
#
# Chapter 02 — Man Pages, Variables & Permissions
# Real commands: man, help, compgen -b, which, type, file, $PATH,
# built-in variables, chmod, stat.

echo "man covers real programs on disk; bash builtins have their own help:"
echo "  man cp        # a real program"
echo "  help history  # history is a shell builtin, not a program"
echo "  compgen -b    # lists every builtin"

echo
echo "which finds programs on \$PATH; it fails on builtins — type works on both:"
which ls
type history

echo
echo "file tells you what kind of file something is:"
file /usr/bin/ls 2>/dev/null || file /bin/ls

echo
echo "\$PATH is a colon-separated list — split it onto separate lines:"
echo "$PATH" | tr ':' '\n' | head -5

echo
echo "A few variables bash sets for you:"
echo "PWD=$PWD"
echo "USER=$USER"
echo "SHELL=$SHELL"

echo
echo "Capture a command's output into a variable:"
now=$(date)
echo "now=$now"

echo
echo "Permissions — chmod +x is what makes a script directly runnable:"
touch demo-script.sh
stat demo-script.sh | head -4
chmod +x demo-script.sh
ls -lha demo-script.sh
rm demo-script.sh
```

# Enfin, les scripts

Vos premiers vrais scripts : lire une saisie de l’utilisateur, regrouper la logique dans des fonctions, ramifier avec if/else, répéter un travail avec des boucles for, et rediriger l’entrée et la sortie entre les commandes et les fichiers.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/03-finally-scripting.sh

```
#!/usr/bin/env bash
#
# Chapter 03 — Finally Scripting
# Real commands: read -p, positional args ($1, $@), functions with
# local, if/[[ ]], for loops (list, range, C-style), I/O redirection.

echo "Take input from the user, or fall back to it if no argument was given:"
if [[ -n $1 ]]; then
    name=$1
else
    name='course-student'   # in place of an interactive read -p prompt
fi
echo "hello $name!"

echo
echo "A function using 'local' so it doesn't leak into the caller's scope:"
my-func() {
    local arg1=$1
    echo "hello from my function, arg1 is $arg1"
}
my-func "$@"

echo
echo "Return codes are separate from printed output:"
check-even() {
    (( $1 % 2 == 0 ))
}
check-even 4 && echo "4 is even (exit code: $?)"

echo
echo "The three for-loop shapes covered in this lesson:"
for name in foo bar baz; do echo "name is $name"; done
for c in {a..c}; do echo "c is $c"; done
for ((i = 0; i < 3; i++)); do echo "i is $i"; done

echo
echo "Reading lines piped in from another command:"
printf 'one\ntwo\n' | while read -r line; do
    echo "got line: $line"
done
```

# Instructions case, tableaux et substitutions

L’instruction case offre une alternative plus claire aux longues chaînes if/elif. Ce chapitre présente aussi les tableaux indexés et associatifs, la variable IFS qui contrôle le découpage en mots, la substitution de commande, les expressions arithmétiques et la substitution de processus.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/04-case-arrays-substitution.sh

```
#!/usr/bin/env bash
#
# Chapter 04 — Case Statements, Arrays & Substitution
# Real commands: case/esac, indexed & associative arrays, IFS,
# $(...) command substitution, $((...)) arithmetic, <(...) process sub.

for name in dave bill anyone; do
    case "$name" in
        d*)  echo "matched d*: $name" ;;
        b*)  echo "matched b*: $name" ;;
        *)   echo "matched *: $name"  ;;
    esac
done

echo
echo "Indexed array — note *the difference* between [*] and [@]:"
array=(foo bar baz)
echo "element 0: ${array[0]}"
echo "last element: ${array[-1]}"
echo "all (as one word): ${array[*]}"
echo "all (as separate words): ${array[@]}"

echo
echo "Associative array — string keys instead of positions:"
declare -A capitals=([France]=Paris [Japan]=Tokyo)
for key in "${!capitals[@]}"; do
    echo "$key is ${capitals[$key]}"
done

echo
echo "Command substitution — prefer \$(...) over legacy backticks:"
thing=$(whoami)
echo "thing is $thing"

echo
echo "Arithmetic expression:"
i=0
((i++))
echo "i is $i"
echo "2 + 2 = $((2 + 2))"

echo
echo "Process substitution treats a command's output like a file:"
diff <(printf 'a\nb\n') <(printf 'a\nc\n') || true
```

# Traitement de texte

Les outils classiques de traitement de texte d'Unix : cut pour extraire des champs, tr pour traduire des caractères, sed pour l'édition de flux, awk pour le traitement par colonnes, grep pour filtrer selon un motif, et find pour localiser des fichiers dans une arborescence.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/05-text-processing.sh

```
#!/usr/bin/env bash
#
# Chapter 05 — Text Processing
# Real commands: cut, tr, sed, awk -F, grep -c, find -type/-name/-exec.

echo "name,age,city" > people.csv
echo "Alice,30,Boston" >> people.csv
echo "Bob,25,Denver" >> people.csv

echo "cut -d',' -f2 — pull just the age column:"
cut -d',' -f2 people.csv

echo
echo "sed — swap Boston for a placeholder:"
sed 's/Boston/[city]/' people.csv

echo
echo "awk -F',' — print field 1 and field 3, skipping the header:"
awk -F',' 'NR > 1 { print $1, $3 }' people.csv

echo
echo "grep -c — count matching lines instead of printing them:"
grep -c 'Alice' people.csv

rm people.csv

echo
echo "find — locate files by type and name, then act on each match:"
find . -maxdepth 1 -type f -name "*.sh"
find . -maxdepth 1 -type f -name "*.sh" -exec echo "found: {}" \;
```

# Arguments Bash, statut des tubes et chronométrage

Lire les arguments passés à votre script, inspecter le code de retour de chaque étape d’un tube avec PIPESTATUS (et pas seulement la dernière), et mesurer le temps d’exécution d’une commande.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/06-arguments-pipes-timing.sh

```
#!/usr/bin/env bash
#
# Chapter 06 — Bash Arguments, Pipe Status & Timing
# Real commands: bash -x (trace), bash -n (syntax check), bash -u
# (error on unset vars), PIPESTATUS, time.

cat > sample.sh << 'INNER_EOF'
#!/usr/bin/env bash
first_name='john'
last_name='eddy'
echo "Hello $first_name $last_name!"
INNER_EOF

echo "bash -n only checks syntax, it does not run the script:"
bash -n sample.sh && echo "syntax OK"

echo
echo "bash -x traces every command as it executes:"
bash -x sample.sh

rm sample.sh

echo
echo "PIPESTATUS holds the exit code of every stage in a pipeline, not"
echo "just the last one that \$? would give you:"
true | false | true
echo "PIPESTATUS: ${PIPESTATUS[@]}"

echo
echo "time reports real/user/sys time for a command:"
time sleep 0.2
```

# Chargement de code (source), sous-shells et retour ou sortie

La commande source charge le code d’un autre fichier dans votre shell courant, tandis que les accolades { } et les parenthèses ( ) déterminent si les commandes s’exécutent dans le shell courant ou dans un sous-shell isolé. Ce chapitre explique aussi la différence entre le code de retour numérique d’une fonction et ce qu’elle affiche.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/07-sourcing-and-subshells.sh

```
#!/usr/bin/env bash
#
# Chapter 07 — Sourcing Code, Subshells & Return vs. Output
# Real commands: source, { } vs ( ), capturing stdout vs stderr
# separately when calling a function.

cat > lib.sh << 'INNER_EOF'
greet() {
    local name=$1
    echo "hello $name!"
}
INNER_EOF

source lib.sh      # loads greet() into THIS shell
greet dave
rm lib.sh

echo
echo "{ } runs in the current shell; ( ) runs in an isolated subshell:"
{ set_in_brace="yes"; }
echo "set_in_brace=$set_in_brace"
( set_in_paren="yes" )
echo "set_in_paren='${set_in_paren:-unset}'"

echo
echo "A function's stdout and stderr can be redirected independently:"
my-func() {
    echo "this goes to stdout" >&1
    echo "this goes to stderr" >&2
}
output=$(my-func 2>/dev/null)   # keep stdout, throw away stderr
echo "captured: $output"
```

# Expansion des paramètres et des tableaux

La syntaxe d’expansion des paramètres de Bash concentre beaucoup de puissance en peu d’espace : valeurs par défaut pour les variables non définies, suppression de préfixes et de suffixes dans les chaînes, recherche et remplacement, et extraction de tranches de tableaux par indice.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/08-parameter-and-array-expansion.sh

```
#!/usr/bin/env bash
#
# Chapter 08 — Parameter & Array Expansion
# Real commands: case conversion (^ , ,), default/required values
# (:-  ?), replacements (# ## % // ), substrings, array slicing.

s='dave eddy'
echo "uppercase first letter: ${s^}"
echo "uppercase everything:   ${s^^}"

echo
name=${1:-dave}
echo "default if unset: $name"

path=/home/dave/data.txt
echo "strip shortest prefix:  ${path#*/}"
echo "strip longest prefix:   ${path##*/}"
echo "strip shortest suffix:  ${path%/*}"
echo "replace first match:    ${path/a/o}"
echo "replace all matches:    ${path//a/o}"

echo
echo "Substrings by offset and length:"
echo "${s:0:4}"
echo "${s: -4}"

echo
echo "Loop a string one character at a time using substring expansion:"
len=${#s}
for ((i = 0; i < len; i++)); do
    printf '%s' "${s:i:1}"
done
echo

echo
echo "Array expansion works the same way, applied to every element:"
arr=(foo bar baz)
printf '<%s>\n' "${arr[@]/a/1}"
```

# Globbing (motifs de noms de fichiers)

Les jokers comme * et ? permettent de désigner des noms de fichiers sans les énumérer un par un. Ce chapitre ajoute le globbing étendu pour des motifs plus expressifs, ainsi que des options du shell comme nullglob qui modifient le comportement d’un motif sans correspondance.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/09-globbing.sh

```
#!/usr/bin/env bash
#
# Chapter 09 — Globbing
# Real commands: *, ?, [..] character classes, extglob's !( ) +( ) @( ),
# and the nullglob/dotglob/globstar shell options.

mkdir -p files
touch files/foo.txt files/foo.jpg files/bar.txt files/baz.jpg

echo "Basic wildcard — every .txt file:"
printf '%s\n' files/*.txt

echo
echo "? matches exactly one character:"
printf '%s\n' files/ba?.txt

echo
echo "[..] matches one character from a set:"
printf '%s\n' files/ba[rz].txt

echo
echo "Extended globbing (needs extglob enabled) — everything except .txt:"
shopt -s extglob
printf '%s\n' files/!(*.txt)
shopt -u extglob

echo
echo "nullglob — a non-matching glob expands to nothing instead of the"
echo "literal pattern string:"
shopt -s nullglob
printf '<%s>\n' files/*.md
shopt -u nullglob

rm -rf files
```

# Expansion des accolades

L’expansion des accolades génère du texte à partir de listes séparées par des virgules ou de plages numériques avant même que le shell ne consulte le système de fichiers — un moyen rapide de produire des séquences de noms de fichiers ou de valeurs.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/10-brace-expansion.sh

```
#!/usr/bin/env bash
#
# Chapter 10 — Brace Expansion
# Real commands: {a,b,c} comma lists, {1..N} numeric ranges, and
# combining braces with a shared filename prefix.

echo "Comma list:"
echo {hello,world}

echo
filename='my-file'
echo "Prefix + comma list, useful for generating related filenames:"
echo "$filename."{txt,jpg,mov}

echo
echo "Numeric range with a step:"
echo {1..20..5}

echo
echo "Combined with globbing — braces don't require the files to exist,"
echo "unlike a glob pattern:"
mkdir -p files
touch files/{foo,bar}.txt
printf '<%s>\n' files/{foo,bar}.txt
rm -rf files
```

# printf, formatage des dates et expressions régulières

printf offre un contrôle précis du format de sortie, la commande date formate les horodatages comme vous le souhaitez, [[ =~ ]] compare des chaînes à des expressions régulières, et mapfile lit l'entrée directement dans un tableau.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/11-printf-dates-regex.sh

```
#!/usr/bin/env bash
#
# Chapter 11 — printf, Date Formatting & Regular Expressions
# Real commands: printf format specifiers (%s %d %5s %-5s %05d),
# date +FORMAT, printf '%(FORMAT)T', regex with [[ =~ ]], mapfile.

echo "printf needs a format string — passing data directly is a bug:"
s='hello world'
printf '%s\n' "$s"

echo
echo "Width and padding:"
printf '%5s|%-5s|\n' hi hi
printf '%05d\n' 55

echo
echo "Date formatting, two ways:"
datefmt='%Y-%m-%d %H:%M:%S'
date "+$datefmt"
printf "%($datefmt)T\n" -1

echo
echo "Matching a pattern with a regex:"
email='user@example.com'
if [[ "$email" =~ ^[^@]+@[^@]+\.[a-z]+$ ]]; then
    echo "'$email' looks like an email"
fi

echo
echo "mapfile reads lines straight into an array:"
printf 'one\ntwo\nthree\n' > lines.txt
mapfile -t lines < lines.txt
echo "line 2: ${lines[1]}"
rm lines.txt
```

# Opérateurs de test

La commande de test classique [ ], le [[ ]] propre à Bash avec une analyse plus sûre, et la commande test elle-même — ainsi que les options de test des chaînes et des fichiers (-z, -f, -d, etc.) dont vous aurez constamment besoin dans les conditions.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/12-test-operators.sh

```
#!/usr/bin/env bash
#
# Chapter 12 — Test Operators
# Real commands: help '[[', help '[', help test, and quoted
# ANSI-C ($'...') strings.

touch demo.txt

[ -f demo.txt ]; echo "[ -f ] exit code: $?"
[[ -f demo.txt && -r demo.txt ]] && echo "[[ ]]: demo.txt exists and is readable"
test -f demo.txt && echo "test: demo.txt exists"

rm demo.txt

echo
echo "Plain echo doesn't interpret backslash escapes by default:"
echo 'line1\nline2'

echo
echo "\$'...' (ANSI-C quoting) does interpret them:"
echo $'line1\nline2'
```

# Signaux (trap) et tubes nommés

trap intercepte des signaux comme Ctrl+C pour que votre script puisse faire le ménage avant de se terminer, et les tubes nommés (FIFO) permettent à deux processus distincts d’échanger des données via un chemin sur le disque qui se comporte comme un fichier.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/13-traps-and-named-pipes.sh

```
#!/usr/bin/env bash
#
# Chapter 13 — Trap Signals & Named Pipes
# Real commands: trap ... exit, mkfifo, reading/writing a FIFO path.

cleanup() {
    echo "cleanup function running"
}
trap cleanup exit

echo "script starting"
echo "..."
echo "script done"
# 'cleanup' fires here automatically because of the trap on exit

echo
echo "Named pipes let two processes talk through a path on disk:"
pipe="/tmp/course-pipe-$$"
mkfifo "$pipe"

echo "[client] hello" > "$pipe" &
read -r line < "$pipe"
echo "[reader] got: $line"

rm -f "$pipe"
```

# Affichage dans le terminal et couleurs

Les séquences d’échappement ANSI ajoutent de la couleur et déplacent le curseur pour des indicateurs de progression, et vérifier que stdout est bien un TTY évite que votre script n’écrive des séquences d’échappement brutes dans un fichier journal ou un tube.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/14-terminal-output-and-color.sh

```
#!/usr/bin/env bash
#
# Chapter 14 — Terminal Output & Color
# Real commands: \e[...m ANSI escapes, tput setaf/sgr0, cursor
# save/restore with \e7 and \e8, and the [ -t 1 ] TTY check.

echo -e '\e[1mbold text\e[0m'
echo $'\e[31mred text\e[0m'

echo
echo "tput is a more portable way to reach the same codes:"
tput setaf 2; echo "green via tput"; tput sgr0

echo
echo "Save cursor position, print elsewhere, then restore it:"
printf '\e7'
printf '\e[5;10Hoverwritten text'
printf '\e8'
echo

echo
echo "Only use color if stdout is an actual terminal:"
if [[ -t 1 ]]; then
    echo "stdout is a TTY — color is safe here"
else
    echo "stdout is redirected — skip the escape codes"
fi
```

# PS1 et personnalisation de Bash

L’invite de votre shell n’est qu’une variable (PS1) que vous pouvez personnaliser pour afficher votre utilisateur, la machine et le répertoire, et les raccourcis readline comme Ctrl+R pour chercher dans l’historique rendent l’usage quotidien du terminal plus rapide.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/15-prompt-and-customization.sh

```
#!/usr/bin/env bash
#
# Chapter 15 — PS1 & Customizing Bash
# Real commands: PS1 escape sequences, shopt -s cdspell/autocd,
# conditional aliases for ~/.bashrc.

echo "Current PS1:"
echo "$PS1"

echo
echo "A custom prompt using \u \h \w — user, host, working directory:"
echo '  PS1="\u@\h:\w\$ "'

echo
echo "Two interactive-only shell options worth turning on in .bashrc:"
echo '  shopt -s cdspell   # forgives small typos in cd targets'
echo '  shopt -s autocd    # typing a directory name alone cd'"'"'s into it'

echo
echo "A conditional alias — only defined if the feature is supported:"
echo '  ls --color=auto &>/dev/null && alias ls="ls --color=auto"'

echo
echo "Readline shortcuts worth practicing at the prompt (not scriptable):"
echo "  Ctrl+W   delete one word back"
echo "  Ctrl+U   delete to start of line"
echo "  Ctrl+R   reverse search through history"
```

# Pièges courants

Notes de terrain issues de vrais bogues : pourquoi analyser la sortie de ls échoue avec des noms de fichiers inhabituels, pourquoi les alias ne peuvent pas prendre d’arguments comme les fonctions, et pourquoi la longueur d’une chaîne peut surprendre avec des caractères multi-octets.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/16-common-pitfalls.sh

```
#!/usr/bin/env bash
#
# Chapter 16 — Common Pitfalls
# Real commands: the classic `for f in $(ls)` bug, alias argument
# limitations, and ANSI codes inflating string length.

mkdir -p files
touch files/a files/b "files/d      e"

echo "The fragile way — breaks on filenames containing spaces:"
for f in $(ls files); do
    echo "  bad: $f"
done

echo
echo "The safe way — glob directly, no ls, no word-splitting:"
for f in files/*; do
    echo "  good: $f"
done

rm -rf files

echo
echo "Aliases can't take positional arguments the way functions can:"
shopt -s expand_aliases
alias greet-alias='echo hello \$1'   # \$1 never gets replaced
greet-alias world

greet-func() { echo "hello $1"; }
greet-func world

echo
echo "ANSI color codes count toward string length even though they're"
echo "invisible on screen:"
colored=$'\e[33mHello\e[0m'
echo "visible text is 5 characters, but \${#colored} reports: ${#colored}"
```

# Bombes fork et conclusion

Un regard sur la fameuse bombe fork en une ligne, présentée comme une curiosité du shell à manier avec prudence — expliquée, jamais exécutée — suivi d’un récapitulatif de tout ce que le cours a couvert.

**▼ Voir le script**

○ ○ ○ bash-scripting/scripts/17-forkbombs-and-wrapup.sh

```
#!/usr/bin/env bash
#
# Chapter 17 — Forkbombs & Course Wrap-up
#
# This chapter covers the classic bash forkbomb: a function that
# recursively spawns two copies of itself in the background, forever,
# exhausting process slots or memory. It's a well-known piece of shell
# trivia (search "bash fork bomb" for the one-liner), included here
# for recognition only — this file deliberately does not include or
# execute one.
#
# If you ever want to see the effect safely, do it only in a disposable
# VM or container, and cap your process count first:
#   ulimit -u 100

echo "A forkbomb defines a function that calls itself twice, piped"
echo "together and backgrounded, doubling the process count on every"
echo "generation until the system runs out of room."

echo
echo "That wraps up Part 1 — Bash Scripting."
echo "Next up: Part 2 — R Programming."
```



# Programmation R

17 leçons · Enseignement : Dr. Lema Logamou Seknewna

17 chapitres, de l'installation de R à un premier aperçu de l'ACP, du clustering et de la régression. Contenu du cours adapté d'un cours d'introduction à R du Dr Lema Logamou Seknewna (AIMS / Institut National de la Statistique du Rwanda).

---

Chapitre 00 · 1 sur 17

## Premiers pas

Installer R et RStudio, configurer R Markdown pour produire du HTML, du Word et du PDF, et se procurer les quelques outils supplémentaires (comme tinytex) dont dépendent les chapitres suivants.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/00-getting-started.Rmd

````
---
title: "00 - Getting Started with R"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

## What is R?

R is a statistical programming language created in 1992 by Ross Ihaka and
Robert Gentleman at the University of Auckland, New Zealand, and first
released as version 1.0 in February 2000. Its name comes from the initials
of its creators' first names. R descends from the S language, which was
developed at Bell Laboratories in the mid-1970s. See the
[official R project page](https://www.r-project.org/about.html) for more
history.

## Install R

Download the build for your operating system from CRAN:

* [Windows](https://cloud.r-project.org/bin/windows/base/)
* [macOS](https://cloud.r-project.org/bin/macosx/)
* [Linux (Debian, Fedora/Red Hat, Ubuntu)](https://cloud.r-project.org/bin/linux/)
````

````
## Install RStudio

RStudio (now Posit) is the IDE most R users work in day to day. Download it
from [posit.co/download/rstudio-desktop](https://posit.co/download/rstudio-desktop/),
or use [Posit Cloud](https://posit.cloud/) to run RStudio entirely in a
browser with no install at all.

## Setting up R Markdown

### Required packages

* **tinytex** — a lightweight LaTeX distribution needed to render R Markdown
  documents to PDF.

```{r eval=FALSE}
if (!"tinytex" %in% rownames(installed.packages())) {
  install.packages("tinytex")
}
library(tinytex)
install_tinytex() # installs LaTeX itself
```

* **Rtools** (Windows only) — needed to build packages from source. Download
  from [cran.r-project.org/bin/windows/Rtools](https://cran.r-project.org/bin/windows/Rtools/).

Once these are installed, R Markdown (`.Rmd`) files like the ones in this
course can render to HTML, Word, or PDF directly from RStudio's Knit button.
````

---

Chapitre 01 · 2 sur 17

# Opérateurs

Opérateurs arithmétiques, opérateurs logiques et recherche de valeurs avec match() et %in% — les petites briques de base sur lesquelles repose chaque script ultérieur.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/01-operators.Rmd

````
---
title: "01 - Operators"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

R can act as a calculator directly from the console — every operator below
can be typed and evaluated on the spot.
````

# Arithmetic Operators

### Addition: `+`
```{r}
1 + 1
```

### Subtraction: `-`
```{r}
1 - 1
```

### Multiplication: `*`
```{r}
1 * 1
```

### Division: `/`
```{r}
1 / 1
```

### Modulus (remainder of a division): `%%`
```{r}
1 %% 2
```

### Exponent: `^` or `**`
```{r}
2 ^ 10 # or 2 ** 10
```

### Integer division: `%/%`
```{r}
1035 %/% 3
```

# Logical Operators

### Less than: `<`
```{r}
1 < 1
```

### Less than or equal to: `<=`
```{r}
1 <= 1
```

### Greater than: `>`
```{r}
1 > 1
```

### Greater than or equal to: `>=`
```{r}
1 >= 1
```

### Exactly equal to: `==`
```{r}
"R" == "r"
```

```

R is case-sensitive — `"R"` and `"r"` are different values.

The equality operator can also compare one element against several at once:
```{r}
"Species" == c("Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width", "Species")
```

### Not equal to: `!=`
```{r}
1 != 1
```

### Negation / NOT: `!`
Flips `TRUE` to `FALSE` and vice versa.
```{r}
!TRUE  # or !T
!FALSE # or !F

!(TRUE & FALSE) # this is TRUE
!(FALSE | TRUE) # this is FALSE
```

### AND: `&`
```{r}
TRUE & TRUE
TRUE & FALSE
FALSE & FALSE
```

### OR: `|`
```{r}
TRUE | TRUE
TRUE | FALSE
FALSE | FALSE
```

## Value Matching

`match()` returns the position of the first match of its first argument in
its second argument. Check `?match` for the full documentation.
```{r}
match("Species", c("Sepal.Length", "Sepal.Width", "Petal.Length",
                   "Petal.Width", "Species"))
```

`%in%` checks whether a value exists anywhere in a vector, returning
`TRUE`/`FALSE` instead of a position:
```{r}
"Species" %in% c("Sepal.Length", "Sepal.Width", "Petal.Length",
                 "Petal.Width", "Species")
```

---



# Objets R, affectation et types de données

Affecter des valeurs avec <-, = et ->, les règles pour des noms de variables valides, les types de données de base de R (numeric, integer, complex,
```

character, logical, raw, factor) et la façon de les convertir proprement les uns en les autres.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/02-objects-assignment-data-types.Rmd

````
---
title: "02 - R Objects, Assignment & Data Types"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# Assignment

R accepts three assignment operators: `<-`, `=`, and `->`.

A variable name:

* can begin with a letter or a dot. E.g. `a <- 1`, `0 -> .a`
* cannot contain spaces — use `_` or `.` instead
* can contain numbers, just not as the first character. E.g. `a1 <- 1`

```{r error=TRUE}
v rsion <- 4.3.2 # invalid: a space inside the name breaks the parser
```

```{r}
a <- 1
b <- 2
0 -> .a
a1 <- .a
```

# Data Types

R has these core data types: numeric, integer, complex, character, logical,
raw, and factor.

## Numeric / double
Examples: `10.5`, `55`, `787`, `pi`.
```{r}
PI <- pi; class(PI); typeof(PI)
n <- 55; class(n); typeof(n)
```

## Integer
Append `L` to a literal to declare it as an integer (`1L`, `55L`, `100L`).
```{r}
n <- 55L
class(n)
```
````

## Complex
An example of a complex number is $9 + 3i$, where $i$ is the imaginary part.
Multiplying a real number by `1i` converts it to complex.
```{r}
z <- 9 + 3i
class(z)
typeof(z)

z1 <- a + 1i * b
print(z1)
class(z1)
```

## Character / string
```{r}
string <- "I am Learning R"
class(string)
```
Remember: `"LeaRning"` is different from `"Learning"` — R is case-sensitive.

## Logical / Boolean
```{r}
TRUE  # or T
FALSE # or F
```
A logical value is often the *result* of a test:
```{r}
"LeaRning" == "Learning"
```

## Raw
```{r}
text <- "I am learning R."
(raw_text <- charToRaw(text))
class(raw_text)
```
Converting raw back to text:
```{r}
rawToChar(raw_text)
```

## Factors
Factors represent qualitative/categorical data — colors, pass/fail, movie ratings — and are the backbone of statistical modeling in R.
```{r}
Gender <- factor(c("Female", "Male"))
print(Gender)
class(Gender)
```

## Logical, revisited
```{r}
v <- TRUE
w <- FALSE

class(v); typeof(v)
!v

isTRUE(w)
```

## Creating objects directly

Numeric:
```{r}
n <- 10
x <- numeric(n) # a numeric vector of length n, filled with zeros
print(x)

x[1] <- 2.5
print(x)
```

Integer — note that assigning a decimal silently upgrades it to numeric:
```{r}
n <- 10
x <- integer(n)
print(x)
class(x)

x[1] <- 2.5 # R converts the whole vector to numeric
class(x)
print(x)
```

## Conversions

To convert between data types, use one of the `as.*()` functions from base R. The naming convention is `as.` followed by the target type.

* Numeric to character
```{r}
n <- 55
n_char <- as.character(n)
print(n_char); class(n_char)
```

* Character to numeric
```{r}
s <- "123"
s_num <- as.numeric(s)
print(s_num); class(s_num)
```
A string that isn't a valid number becomes `NA` (with a warning) rather than causing an error:
```{r}
suppressWarnings(as.numeric("abc"))
```

* Factor to character
```{r}
f <- factor(c("low", "high", "medium"))
f_char <- as.character(f)
print(f_char); class(f_char)
```

* Character to factor
```{r}
level <- "medium"
level_fct <- as.factor(level)
print(level_fct); class(level_fct)
```

# Structures de données I : vecteurs et matrices

Construire des vecteurs avec seq(), rep() et sample(), puis passer à l'algèbre matricielle — transposée, inverse, résolution de systèmes linéaires et valeurs propres — pour finir par une ACP faite à la main sur le jeu de données iris.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/03-vectors-and-matrices.Rmd

````
---
title: "03 - Data Structures I: Vectors & Matrices"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

R's most-used data structures are vectors, matrices, arrays, lists, factors,
and data frames. This chapter covers the first two.

# Scalars and Vectors (1D)

A scalar is any single number; a vector is a collection of values of the
same type (and can represent a sequence).

Mixing types inside `c()` forces R to coerce everything to one common type:
```{r}
v <- c(1, "R", TRUE, FALSE, NA)
print(v)
class(v) # everything becomes character (NA is compatible with any type)
```

```{r}
v2 <- c(1, 4, 8, FALSE, TRUE, FALSE, FALSE, TRUE, "R" == "r")
print(v2)
class(v2) # everything becomes numeric: FALSE -> 0, TRUE -> 1
```

## Sequences with `seq()`
```{r}
x <- seq(0, 2 * pi, length.out = 90)
length(x)
unique(round(diff(x), 5)) # the common step size

head(x, 5); tail(x)
range(x)
```

```{r}
````
````

```r
diff(c(4, 6))
diff(range(x)) # same as max(x) - min(x)
```

Indexing with a non-integer floors to the nearest integer:
```{r}
x[11.8] == x[11]
as.integer(9.123486785) # what R does internally with a fractional index
```

The length of a vector:
```{r}
length(x)
a <- 9
length(a) # a scalar is just a vector of length 1
```

## Repetition with `rep()`
```{r}
rep("I learn R", 3)
rep(c(0, 1), 5)
rep(c(0:1), c(3, 3))
rep(c(0:1), each = 3)
```

## Random sampling
```{r}
set.seed(123) # fixes the randomness for reproducibility
y <- sample(0:1, size = 200, replace = TRUE, prob = c(1/3, 1 - 1/3))
sum(y == 0)   # count of zeros
sum(y == 1)   # count of ones
```

`table()` counts occurrences for you:
```{r}
table(y)
```

`prop.table()` converts counts to proportions:
```{r}
prop.table(table(y))
mean(y == 0) # a shortcut: mean() of a logical vector is a proportion
```

## A small survey example

```{r}
Gender <- c("Female", "Male", "Male", "Male", "Female", "Female", "Male",
"Male", "Female", "Female", "Male", "Male", "Male", "Male", "Female",
"Female", "Male", "Female", "Female", "Male", "Female", "Male",
"Male", "Male", "Male", "Female", "Femaale", "Male", "Female",
"Male", "Male", "Female", "Male", "Female", "Malle", "Male", "Female",
"Female", "Femmale", "Female", "Female", "Female", "Female", "Male",
"Male", "Male", "Male")

sum(Gender == "Female")
mean(Gender == "Female")
table(Gender) # note the 3 misspelled entries
```

Cleaning up the typos:
```{r}
Gender[Gender == "Femaale"] <- "Female"
```

```r
Gender[Gender == "Femmale"] <- "Female"
Gender[Gender == "Malle"]   <- "Male"
table(Gender)
```

Sub-setting with `sample()`:
```{r}
set.seed(76)
table(sample(Gender, size = 20))
```

## Vector arithmetic
```{r}
k <- 2
v <- c(1, 0, 3)

k + v; v + k
k - v; v - k
k * v; v * k
v / k
k / v
```

```{r}
k <- 1/2
v <- c(pi, 0, 1, 4)
w <- c(0, pi, pi, 0, 0)

k * v; v * k
v ^ k
v + w
```

# Matrices (2D)

```{r}
(A <- matrix(1:25, ncol = 5, nrow = 5)) # column-major fill by default
(B <- matrix(1:25, nrow = 5, ncol = 5, byrow = TRUE))
```

## Building a matrix by hand
```{r}
v <- c(1, rep(0, 2), 1, 0, rep(1, 3), rep(0, 2), rep(1, 2))
A <- matrix(v, 3, byrow = TRUE)
print(A)
dim(A)
```

The same result using `dim<-`:
```{r}
v <- c(1, rep(0, 2), 1, 0, rep(1, 3), rep(0, 2), rep(1, 2))
dim(v) <- c(4, 3)
v <- t(v)
v
```

A zero matrix:
```{r}
n <- 6; p <- 8
matrix(0, nrow = n, ncol = p)
```

```{r}
```

```r
(A <- matrix(c(1, 0, 2, 5, 2, 1, 4, 2, 0), nrow = 3))
(B <- matrix(c(2, 5, 2, 3, 1, 1, 0, 1, 1), nrow = 3))
```

## Matrices from vectors: `cbind` / `rbind`

```r
v1 <- c(1, 0, 2); v2 <- c(5, 2, 1); v3 <- c(4, 2, 0)
(M1 <- cbind(v1, v2, v3))
(M2 <- rbind(v1, v2, v3))
```

## Matrix operations

Transpose:

```r
(A_T <- t(A))
```

Addition / subtraction:

```r
A + B
A - B
```

Multiplication (`%*%`, not `*`):

```r
ncol(A) == nrow(B)
A %*% B
```

Inverse:

```r
(A_inv <- solve(A))
round(A %*% A_inv) # sanity check: should be the identity matrix
```

## Solving a system of equations

$$
\begin{cases}
2x + 2y &= 0\\
x + 3y &= 2
\end{cases}
$$

```r
A1 <- matrix(c(2, 2, 1, 3), nrow = 2, byrow = TRUE)
b <- c(0, 2)
solve(A1, b)
```

## Eigenvalues / eigenvectors

Eigenvalues $\lambda$ of a square matrix $A$ solve the characteristic equation $\det(A - \lambda I_n) = 0$. They're the mathematical basis of [Principal Component Analysis](https://www.datacamp.com/tutorial/pca-analysis-r), covered in Chapter 15.

```r
ev <- eigen(A)
print(ev$values)

P <- ev$vectors
```

````
round(solve(P) %*% A %*% P, 4) # recovers a diagonal matrix of eigenvalues
```

## Worked exercise: manual PCA on `iris`

1. Remove the last column (`Species`) from `iris`:
```{r}
data(iris)
dta <- iris[, -5]
```

2. Compute the average of each remaining column:
```{r}
avg <- colMeans(dta)
print(avg)
```

3. Center the data by subtracting each column's average:
```{r}
scaled_dta <- scale(dta, center = TRUE, scale = FALSE)
head(scaled_dta, 3)
```

4. Eigen-decompose `t(scaled_dta) %*% scaled_dta`:
```{r}
cov_mat <- t(scaled_dta) %*% scaled_dta
ev <- eigen(cov_mat)
ev$values
```

5. Project onto the first two eigenvectors:
```{r}
P <- ev$vectors[, 1:2]
projection <- scaled_dta %*% P
head(projection)
```

This is exactly what `prcomp()` does under the hood:
```{r}
pca <- prcomp(dta, center = TRUE, scale. = FALSE)
head(pca$x[, 1:2])
```
````

---

Chapitre 04 · 5 sur 17

# Structures de données II : tableaux, listes et data frames

Tableaux multidimensionnels, listes qui réunissent des types mixtes, et data frames — y compris la transformation d’une variable continue d’âge en catégories de tranches d’âge lisibles.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/04-arrays-lists-dataframes.Rmd

```
---
title: "04 - Data Structures II: Arrays, Lists & Data Frames"
subtitle: "Part 2: R Programming"
```
```

````
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# Arrays

Arrays generalize matrices to more than two dimensions.
```{r}
(aRray <- array(1:24, dim = c(3, 4, 2)))
class(aRray)
```

A classic use case is gridded (NetCDF-style) data: longitude as columns,
latitude as rows, and time as the third dimension — each time slice being
its own row x column matrix.
```{r}
dim(aRray)
aRray[1, 1, 2] # row 1, column 1, of the 2nd matrix "slice"
```

# Lists

A list holds objects of different types and sizes together.
```{r}
L <- list()
A <- matrix(0, 5, 6)

L$A <- A
L$my_seq <- sample(10, 5)
L$Bool <- TRUE
L[["iris_head"]] <- head(iris, 3)
length(L)

L[[5]] <- "I am learning R"
names(L)
names(L)[5] <- "String"
names(L)
```

```{r}
mylist <- list("matrix" = A,
               "sequence" = 1:5,
               "Bool" = TRUE)
mylist$matrix
class(mylist[[1]])
mylist[c("matrix", "Bool")]
```

# Data Frames

A data frame is a table of $n$ rows (observations) by $p$ columns
(variables), where each column can hold a different data type.

## Converting a continuous variable into categories
````

```{r}
set.seed(123)
n <- 120
df <- data.frame(
  "ID"  = paste0("Particip_", 1:n),
  "age" = sample(0:120, size = n)
)
head(df, 5)
```

```{r}
brks <- seq(0, 120, by = 10)
df$age_groups <- cut(df$age, breaks = brks, include.lowest = TRUE)
head(df, 5)
```

## Exercise: summarizing the age groups

1. Counts by age group:
```{r}
(counts <- table(df$age_groups))
```

2. A barplot of the age groups:
```{r fig.height=4, fig.width=7}
x <- barplot(counts, col = "purple", main = "Distribution of ages", xaxt = "n")
text(x = x, y = -1.5, cex = 0.8, xpd = TRUE, srt = 45, labels = names(counts))
```

3. A histogram of raw ages:
```{r fig.height=4, fig.width=7}
hist(df$age, main = "Histogram of ages", xlab = "age", col = "pink", breaks = 12)
```

4. IDs of participants in the (70, 80] age group:
```{r}
df$ID[df$age_groups == "(70,80]"]
```

```{r}
l <- split(df, df$age_groups)
names(l)
l$`(70,80]`$ID
```

## Creating a data frame directly
```{r}
df2 <- data.frame(x = rnorm(10), y = rpois(10, 2))
head(df2, 3)
```

---



# Nettoyage des données : valeurs manquantes et valeurs aberrantes

Repérer et remplir les NA, récupérer des nombres piégés dans des chaînes désordonnées, et signaler les valeurs aberrantes avec la méthode de l’écart interquartile (IQR) avant de les imputer.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/05-data-cleaning.Rmd

````
---
title: "05 - Data Cleaning: Missing Values & Outliers"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# Missing Values (`NA`)

```{r}
x <- c(NA, 1, 2, NA, 3, NA, 3.55)
which(is.na(x))  # which positions are missing
which(x >= 2)    # which positions are >= 2 (NAs drop out silently)
which(!is.na(x)) # which positions are NOT missing
```

Replacing missing values with the mean of the non-missing ones:
```{r}
x[is.na(x)] <- mean(x, na.rm = TRUE)
print(x)
```

## NAs introduced by coercion

Converting a messy character vector straight to numeric turns anything
non-numeric into `NA`:
```{r}
x <- c("2", "1", "2", "7", "$8", "3", "2.5", "9", "2,7")
y <- suppressWarnings(as.numeric(x))
print(y)

id <- which(is.na(y))
id
```

Cleaning the offending values with `gsub()` before converting:
```{r}
x <- c("2", "1", "2", ",7", "$8", "3", "&2.5", "9", "2,7")
x <- gsub("\\$|\\&", "", x)   # drop currency symbols
x <- gsub(",", ".", x)        # normalize decimal commas to dots
y <- as.numeric(x)
print(y)
```

# Outlier Detection

## The Inter-Quartile Range (IQR) method

A value outside the interval below is treated as an outlier:
````

$$[Q_1 - 1.5\times IQR,\ Q_3 + 1.5\times IQR]$$

where $Q_1$/$Q_3$ are the 25th/75th percentiles and $IQR = Q_3 - Q_1$.

```{r fig.height=4, fig.width=5}
x <- c(19.103, 19.632, 22.494, 20.113, 10.5, 22.744, 20.737, 17.976,
       18.901, 192.87, 21.959, 0.001, 20.641, 20.177, 19.111, 22.859,
       207.97, 16.853, 21.122, 19.244, 18.291, 19.651, 18.358, 18.834, 19)

x_copy <- x
boxplot(x_copy, main = "Before handling outliers")
```

```{r}
Q <- as.numeric(quantile(x, probs = c(0.25, 0.5, 0.75)))
d <- as.data.frame(t(Q))
names(d) <- paste0("Q", 1:length(Q))
d$IQR      <- d$Q3 - d$Q1
d$LowerLim <- d$Q1 - 1.5 * d$IQR
d$UpperLim <- d$Q3 + 1.5 * d$IQR
d
```

Flagging every value as an outlier or not:
```{r}
(d_out <- data.frame(x = x, outlier = x < d$LowerLim | x > d$UpperLim))
```

Two equivalent ways to pull out just the outliers:
```{r}
x[d_out$outlier]                                  # using our own table
(bxpt <- boxplot(x, range = 1.5, plot = FALSE))$out # using boxplot()'s own $out
```

## Imputing the outliers

Treat outliers as missing, then fill them with the mean of everything else:
```{r}
x[d_out$outlier] <- NA
x[is.na(x)] <- mean(x, na.rm = TRUE)
```

```{r fig.height=4, fig.width=7}
par(mfrow = c(1, 2))
boxplot(x_copy, main = "Before\nhandling outliers", cex.main = 0.8)
boxplot(x, main = "After\nhandling outliers", col = "green", cex.main = 0.8)
```

No more outliers remain after imputation.

---



# Graphiques et simulations de lois de probabilité

Graphiques de base de R — barres, secteurs, histogramme, nuage de points — puis simulation de lois uniforme, binomiale, gaussienne, exponentielle et de Poisson, ainsi que l’ajustement et l’interprétation d’un modèle linéaire simple.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/06-charts-and-distributions.Rmd

````
---
title: "06 - Charts & Distribution Simulations"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# Base R Charts

## Bar chart
```{r fig.height=4, fig.width=7}
frequencies <- c(Female = 35, Children = 25, Male = 40)
barplot(frequencies, cex.names = 0.8,
        names.arg = names(frequencies),
        col = c("purple", "salmon", "pink"))
```
`cex.names` shrinks the axis labels; lower it further if labels get cut off.

## Pie chart
```{r fig.height=4, fig.width=8}
par(mfrow = c(1, 2))
pie(frequencies, cex = 0.8,
    labels = paste0(names(frequencies), " (", frequencies, ")"),
    col = c("purple", "salmon", "pink"))
pie(frequencies, cex = 0.8,
    labels = paste0(names(frequencies), " (", frequencies, ")"),
    col = rainbow(length(frequencies)))
```

## Histograms and density
```{r fig.height=4, fig.width=8}
set.seed(12092024)
x <- sample(1:120, size = 1000, replace = TRUE)
par(mfrow = c(1, 2))
hist(x, probability = TRUE) # densities instead of raw counts
plot(density(x), col = "salmon", main = "density of x")
```

```{r fig.height=4, fig.width=6}
with(iris, hist(Sepal.Length, probability = TRUE))
lines(density(iris$Sepal.Length), col = "red")
```

## Scatter plot
```{r fig.height=4, fig.width=6}
x <- seq(0, 2 * pi, length.out = 50)
y <- sin(x)
plot(x, y, pch = 16, col = "blue")
```
````

**Exercise:** scatter plot of `Petal.Length` vs `Petal.Width`, colored by species, with a fitted regression line.

```{r fig.height=4, fig.width=6}
f <- Petal.Length ~ Petal.Width
lr <- lm(f, data = iris)
plot(f, data = iris, col = iris$Species, pch = 16)
abline(lr, col = "blue", lwd = 2, lty = 3)
legend("topleft", legend = unique(iris$Species),
       col = unique(iris$Species), pch = 16, bty = "n")
```

# Distribution Simulations

## Uniform distribution

$$
f_X(x) = \begin{cases} \frac{1}{b-a} & \text{if } x \in[a, b] \\
0 & \text{else}\end{cases}
$$

```{r fig.height=4, fig.width=6}
set.seed(123)
x <- runif(1000)
hist(x, probability = TRUE, xlab = NULL,
     main = "Histogram of a uniform distribution", col = "turquoise")
lines(density(x), col = "red", lwd = 2, lty = 3)
abline(v = mean(x), col = "blue", lty = 2, lwd = 2)
legend("topleft", lty = c(2, 2), col = c("red", "blue"),
       legend = c("Density", "Average"))
```

Testing whether a sample looks uniform vs. normal:

```{r fig.height=4, fig.width=5}
ks.test(x, "punif") # p > 0.05 supports "looks uniform"
qqnorm(x)
qqline(x, col = "red")
```

## Binomial distribution

$$
P(X = k) = {n \choose k}p^k(1-p)^{n-k}
$$

```{r fig.height=4, fig.width=8}
rbinom_dist <- rbinom(n = 10000, size = 1, prob = 0.3)
par(mfrow = c(1, 2))
pie(table(rbinom_dist))
barplot(table(rbinom_dist))
```

A histogram isn't well suited to discrete data like this — bar/pie charts work better.

## Gaussian (normal) distribution

$$
f_X(x) = \frac{1}{\sigma\sqrt{2\pi}}\exp\left\{-\frac{1}{2\sigma^2}(x-\mu)^2\right\}
$$

```{r fig.height=4, fig.width=6}
set.seed(13092024)
gauss_dist <- rnorm(1000, mean = 0, sd = 1)
```

```r
hist(gauss_dist, probability = TRUE, breaks = 30, xlab = NULL,
     main = "Histogram of a standard normal distribution", col = "turquoise")
lines(density(gauss_dist), col = "red", lwd = 2, lty = 2)
abline(v = mean(gauss_dist), col = "blue", lty = 2, lwd = 2)
```

```{r}
mean(gauss_dist)
sd(gauss_dist)
```

## Simulating a linear relationship

$$y_i = \alpha + \beta x_i + \varepsilon_i,\quad \varepsilon_i \sim \mathcal{N}(0,1)$$

```{r}
set.seed(123)
x <- rnorm(1000, mean = 5, sd = 1.5)
error_term <- rnorm(1000)
z <- 4 + 8 * x + error_term # true relationship, known only because we simulated it

df <- data.frame(x, z)
head(df, 5)
```

Fitting a simple linear model:
```{r fig.height=5, fig.width=7}
LR <- lm(z ~ x, data = df)
coefs <- coefficients(LR)
summary(LR)

par(mfrow = c(2, 2))
plot(LR) # standard diagnostic plots
```

Estimated model: $\hat{y} =$ `r round(coefs[[1]], 2)` + `r round(coefs[[2]], 2)`
$x$. Predicting for a new value, $x = 15$:
```{r}
new_data <- data.frame(x = 15)
(new_data$predicted <- predict(object = LR, newdata = new_data))
```

```{r fig.height=4, fig.width=6}
plot(predicted ~ x, data = new_data, col = "darkgreen", xlim = c(0, 15),
     ylim = c(0, new_data$predicted), pch = 16, ylab = "z")
points(z ~ x, data = df, col = "darkorange", pch = 20)
abline(LR, col = "blue")
legend("topleft", c("Actual", "Predicted"),
       col = c("darkorange", "darkgreen"), pch = c(20, 16), bty = "n")
```

## Exponential distribution
```{r fig.height=4, fig.width=6}
set.seed(13092024)
exp_dist <- rexp(1000, rate = 1)
hist(exp_dist, probability = TRUE, xlab = NULL,
     main = "Histogram of exponential distribution", col = "turquoise")
lines(density(exp_dist), col = "red", lwd = 2, lty = 2)
abline(v = mean(exp_dist), col = "blue", lty = 2, lwd = 2)
```

## Poisson distribution
```{r fig.height=4, fig.width=6}
```

```
set.seed(13092024)
pois_dist <- rpois(1000, lambda = 2.5)
hist(pois_dist, probability = TRUE, main = "Histogram of Poisson distribution",
     col = "turquoise", xlab = NULL)
lines(density(pois_dist), col = "red", lwd = 2, lty = 2)
abline(v = mean(pois_dist), col = "blue", lty = 2, lwd = 2)
```

---



# Structures de contrôle : conditions et boucles

if/else, for, while et repeat, travaillés à travers un système de notation d'étudiants, un jeu de dés à séries, et une boucle sur un dossier de fichiers pour tous les importer d'un coup.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/07-flow-control.Rmd

````
---
title: "07 - Flow Control: Conditionals & Loops"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# `if` / `else`

```
if (condition) {
  ## code to run
}
```

```{r}
x <- 3
if (x < 4) {
  print(TRUE)
} else {
  print(FALSE)
}

# one-liner form
if (x < 4) print(TRUE) else print(FALSE)

# vectorized form: ifelse() works element-by-element on a whole vector
x <- c(3, 4, 6, 7, 0, 4)
ifelse(x < 4, TRUE, FALSE)
```
````

## Nested conditions: grading example

```{r}
set.seed(123)
marks <- round(rnorm(50, mean = 60, sd = 15), 2)
```

Categorize each mark: Fail [0-60), Pass [60-70), Good Pass [70-80),
Very Good Pass [80-85), Distinction [85+].

**Approach 1 — iterate over values directly:**
```{r}
category <- character(0)
for (m in marks) {
  if (m >= 0 & m < 60) {
    category <- c(category, "Fail")
  } else if (m >= 60 & m < 70) {
    category <- c(category, "Pass")
  } else if (m >= 70 & m < 80) {
    category <- c(category, "Good Pass")
  } else if (m >= 80 & m < 85) {
    category <- c(category, "Very Good Pass")
  } else {
    category <- c(category, "Distinction")
  }
}
head(df_marks <- data.frame(marks, category), 5)
```

**Approach 2 — iterate over indices:**
```{r}
category2 <- character()
for (i in seq_along(marks)) {
  if (marks[i] >= 0 & marks[i] < 60) {
    category2[i] <- "F"
  } else if (marks[i] >= 60 & marks[i] < 70) {
    category2[i] <- "P"
  } else if (marks[i] >= 70 & marks[i] < 80) {
    category2[i] <- "GP"
  } else if (marks[i] >= 80 & marks[i] < 85) {
    category2[i] <- "VGP"
  } else {
    category2[i] <- "D"
  }
}
df_marks$category2 <- category2
head(df_marks, 5)
```

**Approach 3 — vectorized with nested `ifelse()`:**
```{r}
category3 <- ifelse(marks < 60, "Fail",
              ifelse(marks < 70, "Pass",
               ifelse(marks < 80, "Good Pass",
                ifelse(marks < 85, "Very Good Pass", "Distinction"))))
all(df_marks$category == category3)
```

# `for` Loops

```{r}
m <- 10
```

```
for (i in 1:m) print(i)
```

```{r}
for (i in 1:10) {
  if (i %% 3 == 0) {
    cat(i, "is a multiple of 3\n")
  } else if (i %% 2 == 0) {
    cat(i, "is a multiple of 2\n")
  } else {
    cat(i, "is neither a multiple of 2 nor 3\n")
  }
}
```

## Exercise 1: odd or even

```{r}
for (i in 1:10) {
  if (i %% 2 == 0) {
    cat(i, "is even\n")
  } else {
    cat(i, "is odd\n")
  }
}
```

## Exercise 2: importing every CSV in a folder

Build the file path from parts using `paste()`/`paste0()`:
```{r}
paste("data", "csv", sep = "/")
paste0("data", "/", "csv")
```

List every CSV in a folder, then load them all into one list:
```{r}
dir.create("demo_csv", showWarnings = FALSE)
write.csv(head(iris, 3), "demo_csv/iris_part.csv", row.names = FALSE)
write.csv(head(mtcars, 3), "demo_csv/mtcars_part.csv", row.names = FALSE)

file_names <- dir("demo_csv", pattern = "\\.csv$", full.names = TRUE)
file_names

data_list <- list()
for (f in file_names) {
  data_list[[basename(f)]] <- read.csv(f)
}
str(data_list)
unlink("demo_csv", recursive = TRUE) # clean up the demo files
```

# `while` Loops

```
while (condition) {
  ## code to run
  ## something that eventually makes condition FALSE
}
```

```{r}
i <- 0
```

```
while (i <= 10) {
  print(i * 2)
  i <- i + 10
}
```

## Exercise: dice streak game

Print "YOU WON!" the moment three 5s are rolled in a row.
```{r}
set.seed(42)
streak <- 0
rolls <- 0
repeat {
  rolls <- rolls + 1
  roll <- sample(1:6, 1)
  streak <- if (roll == 5) streak + 1 else 0
  if (streak == 3) {
    cat("YOU WON! (after", rolls, "rolls)\n")
    break
  }
}
```

## Exercise: interactive multiplication table

This one needs a live console, since `readline()` only works in an
interactive R session (not when a script is run non-interactively):
```{r eval=FALSE}
repeat {
  number <- as.numeric(readline(prompt = "Enter a number 0-9: "))
  cat(number, "x 2 table:\n")
  for (i in 1:10) cat(number, "x", i, "=", number * i, "\n")

  again <- readline(prompt = "Continue? (y/n): ")
  if (tolower(again) != "y") break
}
```

# `repeat`

```
i <- 0
repeat {
  ## code to run
  i <- i + 1
  if (something_happens) break
}
```

```{r}
i <- 0
repeat {
  print(i)
  i <- i + 1
  if (i > 5) break
}
```

# La famille des fonctions apply

apply, sapply, lapply, tapply et vapply, des alternatives plus rapides et plus lisibles que d’écrire à chaque fois une boucle explicite.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/08-apply-functions.Rmd

````
---
title: "08 - The Apply Family of Functions"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

The `apply` family replaces explicit loops when you want to run the same
function across rows, columns, or elements of a data structure.

# `apply()`

Applies a function across the margins (rows or columns) of a matrix.
```{r}
A <- matrix(1:4, nrow = 2)
A

avg <- function(x) sum(x) / length(x)
avg(1:10)
```

```{r}
apply(iris[-5], MARGIN = 2, summary) # MARGIN = 2 means "column-wise"
```

# `sapply()`

Like `apply()`, but works over vectors/lists and simplifies the result to
a vector or matrix where possible.
```{r}
sapply(A, sum) # does not respect matrix structure — flattens first
```

```{r}
coef_disp <- function(x) sd(x) / mean(x)
get_range  <- function(x) diff(range(x))

sapply(Filter(is.numeric, iris), get_range)

sapply(Filter(is.numeric, iris), function(x) c(
  min = min(x), max = max(x), std_dev = sd(x),
  Q1 = quantile(x, 0.25), Med = quantile(x, 0.5), Q3 = quantile(x, 0.75),
  disp = coef_disp(x)
````
````

````
))
```

`sapply()` falls back to returning a list when results aren't the same
length:
```{r}
sapply(iris, summary)
```

```{r}
sapply(1:10, function(x) x^2)
```

# `lapply()`

Always returns a list, one element per input element:
```{r}
a <- lapply(iris[-5], mean)
unlist(a)
```

# `tapply()`

Applies a function to subgroups of a vector, split by a factor:
```{r}
tapply(iris$Sepal.Length, iris$Species, mean)
```

# `vapply()`

Like `sapply()`, but requires you to specify the expected output shape —
safer for use inside functions/packages since it fails loudly on a mismatch:
```{r}
vapply(as.list(iris[-5]), quantile,
       FUN.VALUE = c("0%" = 0, "25%" = 0, "50%" = 0, "75%" = 0, "100%" = 0))
```
````

---

Chapitre 09 · 10 sur 17

# Écrire des fonctions en R

Définir ses propres fonctions à partir de zéro, puis les appliquer à deux exercices guidés : remplacer les valeurs manquantes par une statistique choisie, et résoudre une équation du second degré à partir de son discriminant.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/09-writing-functions.Rmd

```
---
title: "09 - Writing Functions in R"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
```
```

````
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# Function Syntax

```
function_name <- function(arg1, arg2, ...) {
  ## code to run
}
```

```{r}
toss_coin <- function() {
  sample(6, size = 1)
}

set.seed(1)
faces <- sapply(1:10, function(i) toss_coin())
faces
```

```{r}
pp <- function(x) return(x + 1)
i <- 1
(i <- pp(i))
```

# Exercises

## 1. Replace missing values with a summary statistic

```{r}
replace_missing <- function(x, fun = mean) {
  x[is.na(x)] <- fun(x, na.rm = TRUE)
  x
}

test_vec <- c(1, 2, NA, 4, NA, 6)
replace_missing(test_vec)
replace_missing(test_vec, fun = median)
```

## 2. Solve a quadratic equation and comment on its discriminant

For $ax^2 + bx + c = 0$:

```
                  ┌─ discriminant > 0 → two distinct real roots
compute disc = b² - 4ac ─┼─ discriminant = 0 → one repeated real root
                  └─ discriminant < 0 → two complex conjugate roots
```

```{r}
solve_quadratic <- function(a, b, c) {
  if (a == 0) stop("a must be non-zero for a quadratic equation")

  disc <- b^2 - 4 * a * c

  if (disc > 0) {
    message("Two distinct real roots")
````
````

```
    roots <- c((-b + sqrt(disc)) / (2 * a), (-b - sqrt(disc)) / (2 * a))
  } else if (disc == 0) {
    message("One repeated real root")
    roots <- -b / (2 * a)
  } else {
    message("Two complex conjugate roots")
    real_part <- -b / (2 * a)
    imag_part <- sqrt(-disc) / (2 * a)
    roots <- c(complex(real = real_part, imaginary = imag_part),
               complex(real = real_part, imaginary = -imag_part))
  }
  roots
}

solve_quadratic(1, -3, 2)  # disc > 0
solve_quadratic(1, 2, 1)   # disc == 0
solve_quadratic(1, 0, 1)   # disc < 0
```
```

---

Chapitre 10 · 11 sur 17

# Packages

Installer des packages depuis le CRAN, les charger avec library() ou pacman, lire leur documentation, et découvrir les verbes essentiels de dplyr — select, filter, arrange, rename, mutate, summarise — pour la manipulation courante des données.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/10-packages.Rmd

```
---
title: "10 - Packages"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

A package is a documented bundle of functions, compiled code, and data.
Packages exist to make one specific job easy.

# Installing a Package

## From CRAN

Check the [full list of CRAN packages](https://cran.r-project.org/web/packages/available_packages_by_name.html).
If a package isn't already installed, install it with `install.packages()`.

```{r}
```

```
head(data.frame(installed.packages())[c(1, 3:5)], 5)
```

The installed-packages matrix has a `Package` column, easy to check with
`installed.packages()[, "Package"]`:
```{r}
all_packages <- installed.packages()[, "Package"]
"pacman" %in% all_packages
```

Install only if missing:
```{r message=FALSE, warning=FALSE}
if (!"pacman" %in% all_packages) {
  install.packages("pacman", repos = "http://cran.us.r-project.org")
}
```

# Loading a Package

`library()` (from base R) makes a package's functions available:
```{r warning=FALSE, message=FALSE}
library(pacman)
```

`pacman::p_load()` loads a whole list of packages at once, installing any
that are missing:
```{r eval=FALSE}
pkg_list <- c("tidyverse", "ggplot2", "lubridate", "flextable")
p_load(pkg_list, character.only = TRUE)
```

## Package documentation

```{r eval=FALSE}
help(package = "pacman")
```
The full documentation is also available under RStudio's
**Files | Plots | Packages | Help** pane.

## Suppressing package startup noise

`tidyverse` (and others) print a message on load that you usually don't
want in a rendered report:
```{r warning=FALSE, message=FALSE}
library(tidyverse)
```

# The `dplyr` Package

`dplyr` is part of `tidyverse` and provides a consistent grammar for data
manipulation, which reads especially well combined with the pipe (`%>%`,
covered in Chapter 12).

The most-used `dplyr` verbs are:

* `select()` — keep a subset of columns
```{r}
iris %>% select(1:4) %>% names()
iris %>% select(-Species) %>% names()
```

* `filter()` — keep a subset of rows matching a condition
```{r}
```

````
iris %>% filter(Species == "setosa", Petal.Length > 1.4) %>% head(3)
```

* `arrange()` — reorder rows
```{r}
iris %>% arrange(desc(Sepal.Length)) %>% head(3)
```

* `rename()` — rename columns
```{r}
iris %>% rename(PetalLength = Petal.Length) %>% names()
```

* `mutate()` — add or transform columns
```{r}
iris %>% mutate(Ratio = Petal.Length / Petal.Width) %>% head(3)
```

* `summarise()` — collapse a group into one row
```{r}
iris %>% group_by(Species) %>% summarise(mean = mean(Petal.Width), .groups = "drop")
```
````

---

Chapitre 11 · 12 sur 17

# Importer des données dans R

Lire des jeux de données intégrés, des fichiers CSV et Excel dans R avec R de base, readr et rio, puis boucler sur tout un dossier de fichiers pour les charger tous en une fois.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/11-importing-data.Rmd

````
---
title: "11 - Importing Data in R"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

> **Note on file paths:** the original course material referenced files
> under `./data/`. Those files aren't part of this repo, so every example
> below either uses a built-in dataset or generates a small temp file on
> the fly — copy the pattern and point it at your own files.

# Built-in Data

R ships with sample datasets ready to use, no import needed.
````
````

```{r}
data(iris)
str(iris)
summary(iris)
ncol(iris)
```

Checking the class of every column, timed with base R (or use the `tictoc`
package's `tic()`/`toc()` for nicer output if you have it installed):
```{r}
dtypes <- c()
start <- Sys.time()
for (i in 1:ncol(iris)) {
  dtypes[i] <- class(iris[[i]])
}
print(Sys.time() - start)
unique(dtypes)
```

Loading a dataset from a package without loading the whole package:
```{r}
data("spam", package = "kernlab")
str(spam[1:5])
```

# CSV Files

Three common ways to read a CSV, from least to most modern:

* `read.csv()` from base R
```{r}
tmp <- tempfile(fileext = ".csv")
write.csv(iris, tmp, row.names = FALSE)

dta <- read.csv(tmp)
head(dta, 3)
```
Notice `Species` comes back as `character` — pass `stringsAsFactors = TRUE`
to `read.csv()` if you want it as a factor instead.

* `read_csv()` from `readr` (loaded together with `tidyverse`)
```{r message=FALSE}
library(readr)
dta <- read_csv(tmp)
head(dta, 3)
```
Always check the messages/warnings `read_csv()` prints — they usually flag
real issues worth fixing.

* `import()` from `rio`, which auto-detects the file type
```{r message=FALSE}
library(rio)
head(import(tmp), 2)
unlink(tmp)
```

## Excel files

`rio::import()` handles `.xlsx` the same way it handles CSV:
```{r message=FALSE}
tmp_xlsx <- tempfile(fileext = ".xlsx")
export(head(iris, 5), tmp_xlsx)
head(import(tmp_xlsx), 2)
```

```
unlink(tmp_xlsx)
```

## Exercise: import every CSV in a folder with a loop

```{r}
dir.create("demo_csv", showWarnings = FALSE)
write.csv(head(iris, 3), "demo_csv/iris_part.csv", row.names = FALSE)
write.csv(head(mtcars, 3), "demo_csv/mtcars_part.csv", row.names = FALSE)

file_names <- dir("demo_csv", pattern = "\\.csv$")
file_names

data_list <- list()
for (f in file_names) {
  data_list[[f]] <- read.csv(file.path("demo_csv", f))
}
str(data_list)
unlink("demo_csv", recursive = TRUE)
```

---

Chapitre 12 · 13 sur 17

# Le pipe et la manipulation de données

Enchaîner les étapes de gauche à droite avec %>% et le pipe natif |>, remodeler les données du format large au format long et inversement avec reshape2/tidyr, et produire des tableaux soignés avec flextable.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/12-pipe-and-dplyr.Rmd

```
---
title: "12 - The Pipe & Data Manipulation"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# The Pipe

Two pipe operators are common in R:

* `%>%` from the `magrittr` package (loaded with `tidyverse`/`dplyr`)
* `|>` the base-R native pipe

RStudio shortcut: **Ctrl+Shift+M**.

Why bother? Because `f(g(h(x)))` reads inside-out, while
```

`x %>% h() %>% g() %>% f()` reads left to right, in the order things
actually happen.

```{r message=FALSE}
library(dplyr)
library(ggplot2)

iris %>% group_by(Species) %>%
  summarise(mean = mean(Petal.Width), .groups = "drop")

Gender <- factor(c("Female", "Male", "Male", "Female", "Female", "Male"))

Gender %>% table() %>% data.frame() %>%
  rename("Gender" = 1, Count = Freq) %>%
  mutate(Rel_freq = round(Count / sum(Count), 2)) %>%
  ggplot(aes(x = Gender, y = Rel_freq)) + geom_col()
```

Without the pipe, the same computation nests inside-out:
```{r}
summarise(group_by(iris, Species), mean = mean(Petal.Width))
```

# Data Manipulation with `tidyverse`

Loading `tidyverse` pulls in several packages at once: `ggplot2`, `tibble`,
`tidyr`, `readr`, `purrr`, `dplyr`, `stringr`, `forcats`, and `lubridate`.
There's no need to load each individually afterward.

## `dplyr`

Grammar-based data manipulation — see Chapter 10 for the core verbs
(`select`, `filter`, `arrange`, `rename`, `mutate`, `summarise`).

## `tibble`

A tibble is a modernized data frame: cleaner printing, no silent partial
matching, and no automatic factor conversion.
```{r}
library(tibble)
tb <- as_tibble(iris)
tb
glimpse(tb)
```

## `reshape2`

Reshaping data between wide and long formats:
```{r message=FALSE, fig.height=3, fig.width=8}
library(reshape2)
library(tidyr)

view1 <- iris %>% group_by(Species) %>%
  summarise_if(is.numeric, list(mean = mean, sd = sd)) %>%
  melt(id.vars = "Species") %>%
  separate(variable, into = c("variable", "metric"), sep = "_")

view1
```

Pivoting back to wide format with `tidyr::pivot_wider()`:
```{r}
view2 <- view1 %>%
```

```
  pivot_wider(names_from = metric, values_from = value) %>%
  mutate(std_error = sd / sqrt(nrow(iris)))
view2
```

```{r fig.height=4, fig.width=8}
view2 %>% ggplot() +
  geom_bar(aes(x = Species, y = mean), stat = "identity", fill = "forestgreen", alpha = 0.5) +
  geom_errorbar(aes(x = Species, ymin = mean - std_error, ymax = mean + std_error),
                width = 0.4, colour = "orange", alpha = 0.9, linewidth = 0.5) +
  facet_wrap(~variable, ncol = 4)
```

# Presenting Tables

## `kableExtra`
Great for styling `knitr::kable()` tables in HTML/PDF reports — see the
[kableExtra cookbook chapter](https://bookdown.org/yihui/rmarkdown-cookbook/kableextra.html).

## `flextable`
Produces polished tables that also render well in Word documents:
```{r message=FALSE, warning=FALSE}
library(broom)
library(flextable)

set.seed(123)
x <- rnorm(100, mean = 5, sd = 1.5)
z <- 4 + 8 * x + rnorm(100)
LR <- lm(z ~ x)

tidy(LR) %>% select(-statistic) %>% flextable() %>% autofit() %>% theme_box()
```

See the [Epi R Handbook tables chapter](https://epirhandbook.com/en/new_pages/tables_presentation.html)
for more `flextable` recipes.

## `officer`
Generates and edits Word/PowerPoint files directly from R — see the
[officeverse guide](https://ardata-fr.github.io/officeverse/) for details.

---

Chapitre 13 · 14 sur 17

# Visualisation : ggplot2 et plotly

Construire des graphiques par couches selon la grammaire des graphiques avec ggplot2, puis les rendre interactifs avec plotly et prêts pour la publication avec ggpubr.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/13-visualization.Rmd

```
---
title: "13 - Visualization: ggplot2 & plotly"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
```

````
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

# `ggplot2`

`ggplot2` builds plots in layers, following the "grammar of graphics."
```{r message=FALSE, fig.height=3, fig.width=8}
library(ggplot2)
library(dplyr)

iris %>%
  ggplot(aes(y = Petal.Length, fill = Species)) +
  geom_boxplot() +
  facet_grid(~Species) +
  theme_minimal()
```

For a full systematic introduction to `ggplot2`, see the
[Data Visualization chapters of R for Data Science](https://r4ds.hadley.nz/data-visualize.html)
or [this ggplot2 bookdown guide](https://bookdown.org/ozancanozdemir/introduction-to-ggplot2/).

# `plotly`

`plotly` turns static plots into interactive, hover-able ones:
```{r message=FALSE, fig.height=4, fig.width=6}
library(plotly)

plot_ly(data = iris %>% count(Species),
        labels = ~Species, values = ~n, type = "pie")
```

`ggpubr::ggpie()` gives a similar publication-ready pie chart built on
`ggplot2`:
```{r message=FALSE, fig.height=4, fig.width=6}
library(ggpubr)

ggpie(data = iris %>% count(Species) %>%
        mutate(label = paste0(Species, " (", round(100 * n / sum(n), 2), "%)")),
      x = "n", label = "label", color = "white",
      fill = "Species", lab.pos = "in")
```

See the [plotly R docs](https://plotly.com/r/) and
[this intro-to-DS plotting chapter](https://bookdown.org/ronsarafian/IntrotoDS/plotting.html)
for more.
````

---



# Expressions régulières

Détecter, extraire et remplacer des motifs de texte avec grepl/grep/sub/gsub de R de base et les équivalents plus cohérents de stringr.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/14-regular-expressions.Rmd

````
---
title: "14 - Regular Expressions"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

A regular expression (regex) is a pattern used to match, extract, or
replace text. R's base functions for this all take a pattern as their
first serious argument and a character vector to search as the second.

```{r}
text <- c("dave@example.com", "call 555-1234", "no match here", "ID: R2024-001")
```

# Detecting Matches

`grepl()` returns `TRUE`/`FALSE` for each element:
```{r}
grepl("[0-9]+", text)
```

`grep()` returns matching positions, or the matching strings themselves
with `value = TRUE`:
```{r}
grep("[0-9]+", text)
grep("[0-9]+", text, value = TRUE)
```

# Replacing Matches

`sub()` replaces only the first match per string; `gsub()` replaces all:
```{r}
sub("[0-9]+", "#", text)
gsub("[0-9]+", "#", text)
```

# Extracting Matches

`regexpr()` finds the *first* match's position; `regmatches()` pulls out
the actual matched text:
```{r}
m <- regexpr("[0-9]+", text)
regmatches(text, m)
```

`gregexpr()` + `regmatches()` extracts *every* match per string:
```{r}
mg <- gregexpr("[0-9]+", text)
````
````

````
regmatches(text, mg)
```

# A Practical Pattern: Validating an Email

```{r}
grepl("^[A-Za-z]+@[A-Za-z]+\\.[a-z]+$", text)
```

# Common Building Blocks

| Pattern | Matches |
|---|---|
| `[0-9]` or `\\d` | any digit |
| `[a-zA-Z]` | any letter |
| `.` | any single character |
| `+` | one or more of the previous token |
| `*` | zero or more of the previous token |
| `^` / `$` | start / end of the string |
| `\\s` | whitespace |
| `\|` | alternation ("or") |

# `stringr` Equivalents

The `stringr` package (part of `tidyverse`) wraps the same regex engine in
more consistent, pipe-friendly functions:
```{r message=FALSE}
library(stringr)
str_detect(text, "[0-9]+")
str_extract(text, "[0-9]+")
str_extract_all(text, "[0-9]+")
str_replace_all(text, "[0-9]+", "#")
```

For deeper coverage, see
[DataCamp's regex in R tutorial](https://www.datacamp.com/tutorial/regex-r-regular-expressions-guide)
or [this regex tutorial](https://jfjelstul.github.io/regular-expressions-tutorial/).
````

---

Chapitre 15 · 16 sur 17

# Introduction à la modélisation : ACP, clustering, KNN et régression

Un premier tour d’horizon de l’apprentissage non supervisé et supervisé : ACP et clustering pour trouver une structure sans étiquettes, puis KNN et régression logistique pour prédire réellement une catégorie.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/15-intro-to-modeling.Rmd

```
---
title: "15 - Intro to Modeling: PCA, Clustering, KNN & Regression"
subtitle: "Part 2: R Programming"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
```
```

````
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

A first tour of the modeling techniques R is most commonly used for,
building on the eigen-decomposition and linear model work from earlier
chapters.

# Principal Component Analysis

Chapter 3 walked through PCA by hand: center the data, eigen-decompose its
covariance matrix, and project onto the top eigenvectors. In practice,
`prcomp()` does all of that for you, plus proper scaling:
```{r}
data(iris)
pca <- prcomp(iris[, -5], center = TRUE, scale. = TRUE)
summary(pca)
```

`Proportion of Variance` shows how much information each component
captures — here the first two components already explain most of the
spread in the data.
```{r fig.height=4, fig.width=6}
plot(pca$x[, 1:2], col = iris$Species, pch = 16,
     xlab = "PC1", ylab = "PC2", main = "iris projected onto PC1/PC2")
legend("topright", legend = levels(iris$Species),
       col = 1:3, pch = 16, bty = "n")
```

# Clustering

## K-means

Partitions data into `k` groups by minimizing within-cluster distance.
Being unsupervised, it never sees `Species` — comparing its output to the
real labels afterward is just for evaluation.
```{r}
set.seed(1)
km <- kmeans(iris[, -5], centers = 3, nstart = 10)
table(cluster = km$cluster, species = iris$Species)
```

## Hierarchical clustering

Builds a tree of nested clusters instead of a single flat partition:
```{r fig.height=4, fig.width=7}
d <- dist(iris[1:20, -5]) # a small subset, so the dendrogram stays readable
hc <- hclust(d, method = "complete")
plot(hc, main = "Hierarchical clustering (first 20 flowers)")

cutree(hc, k = 3) # cut the tree into 3 groups
```

# K-Nearest Neighbor (KNN)

A supervised classifier: label a new point by majority vote among its `k`
closest neighbors in the training data.
```{r message=FALSE}
library(class)
````
````

````
set.seed(1)
idx <- sample(1:nrow(iris), 0.7 * nrow(iris))
train <- iris[idx, ]
test  <- iris[-idx, ]

pred <- knn(train[, -5], test[, -5], cl = train$Species, k = 5)
mean(pred == test$Species) # accuracy on the held-out test set
table(predicted = pred, actual = test$Species)
```

# Simple Linear Regression

Already covered in Chapter 6's simulated-data example
(`z ~ x`, fitted with `lm()`). A quick recap on `iris`:
```{r}
lr <- lm(Petal.Length ~ Petal.Width, data = iris)
coef(lr)
```
Each additional cm of petal width is associated with roughly
`r round(coef(lr)[[2]], 2)` cm more petal length.

# Logistic Regression

For a categorical (here, binary) outcome, swap `lm()` for `glm()` with
`family = binomial`. Predicting *virginica* vs. *versicolor* from sepal
measurements:
```{r}
sub <- subset(iris, Species %in% c("versicolor", "virginica"))
sub$is_virginica <- factor(ifelse(sub$Species == "virginica", 1, 0))

logit <- glm(is_virginica ~ Sepal.Length + Sepal.Width, data = sub, family = binomial)
summary(logit)$coefficients
```

Turning predicted probabilities into class predictions at a 0.5 threshold:
```{r}
pred_prob  <- predict(logit, type = "response")
pred_class <- ifelse(pred_prob > 0.5, 1, 0)

mean(pred_class == sub$is_virginica) # accuracy
table(predicted = pred_class, actual = sub$is_virginica)
```
````

---

Chapitre 16 · 17 sur 17

# LaTeX dans R Markdown

Écrire des formules mathématiques en ligne et en bloc à côté de votre code R — une passerelle naturelle vers la Partie 3, où LaTeX devient le sujet principal.

**▼ Voir le R Markdown**

○ ○ ○ r-programming/chapters/16-latex-in-rmarkdown.Rmd

```
---
title: "16 - LaTeX in R Markdown"
subtitle: "Part 2: R Programming"
output:
  html_document:
```
```

````
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

R Markdown (and Quarto) render LaTeX math directly, both inline and as
display equations — useful for writing up the statistics behind a model
alongside the code that fits it.

# Inline Math

Wrapping an expression in single dollar signs keeps it inline with text:
the variance of a real-valued variable $X = (X_1, \ldots, X_n)$ is written
right in the sentence like this.

Source:
```
The variance of a real-valued variable $X = (X_1, \ldots, X_n)$ is given by:
```

# Display Math

Double dollar signs (or a `$$...$$` block) center the equation on its own
line:

$$
\text{Var}(X) =
\left[\frac{1}{n}\sum_{i=1}^n\left(X_i-\frac{1}{n}
\sum_{i=1}^nX_i\right)^2\right]^\frac{1}{2}
$$

Source:
```
$$
\text{Var}(X) =
\left[\frac{1}{n}\sum_{i=1}^n\left(X_i-\frac{1}{n}
\sum_{i=1}^nX_i\right)^2\right]^\frac{1}{2}
$$
```

# Mixing LaTeX and R

Math and code can sit side by side in the same document — describe a
function mathematically, then define and run it:

$$f(x) = x^3$$

```{r}
cube <- function(x) x^3
cube(3)
```

This is the same LaTeX engine used to render PDF output (via `tinytex`,
set up in Chapter 0), so anything that displays correctly here will also
render correctly in a knitted PDF.
````

# Programmation Python

17 leçons · Enseignement : Dr. Lema Logamou Seknewna

17 chapitres qui reprennent le cours de R, de l'installation de Python à un premier aperçu de l'ACP, du clustering et de la régression. Chaque chapitre est un fichier de type notebook que vous pouvez ouvrir dans Jupyter, VS Code ou Colab ; tous les exemples ont été exécutés avant d'être inclus.

---

Chapitre 00 · 1 sur 17

## Premiers pas

Installer Python, créer un environnement virtuel, choisir où écrire son code (Jupyter, VS Code, Colab) et vérifier que ses outils fonctionnent.

**▼ Voir le code Python**

○ ○ ○ python/chapters/00-getting-started.py

```
# %% [markdown]
# # 00 - Getting Started with Python
# *Python Programming — the Python counterpart of the R course*
#
# ## What is Python?
# Python is a general-purpose programming language created by Guido van Rossum
# and first released in 1991. Its clear, readable syntax and its huge ecosystem
# (data analysis, machine learning, web, automation) have made it one of the
# most widely used languages in the world.
#
# ## Install Python
# Download the installer for your system from <https://www.python.org/downloads/>.
# On Linux, Python is usually pre-installed; if not:
# `sudo apt install python3 python3-pip python3-venv`.
#
# ## Check the installation (in a terminal)
# ```bash
# python3 --version
# pip --version
# ```
#
# ## Work in a virtual environment
# A virtual environment keeps the packages of one project separate from another.
# ```bash
# python3 -m venv .venv
# source .venv/bin/activate          # Windows: .venv\Scripts\activate
# pip install numpy pandas matplotlib scipy scikit-learn jupyterlab
# ```
#
# ## Where to write code
# * **JupyterLab / Jupyter Notebook** — notebooks mix text, code and results.
# * **VS Code** with the Python extension.
# * **Google Colab** — nothing to install, runs in the browser.
```

```
#
# These chapters are written in the "percent" format: every line that starts
# with `# %%` begins a new cell. Jupyter (with Jupytext), VS Code and Spyder open
# them as notebooks, and they also run as ordinary scripts with
# `python3 chapter.py`. A ready-made `.ipynb` version of each chapter is
# provided next to it.

# %%
# Run this cell to check your setup.
import sys

print("Python", sys.version.split()[0])

packages = {"numpy": "numpy", "pandas": "pandas", "matplotlib": "matplotlib",
            "scipy": "scipy", "sklearn": "scikit-learn"}
for module_name, pip_name in packages.items():
    try:
        module = __import__(module_name)
        print(f"{module_name:12s} {module.__version__}")
    except ImportError:
        print(f"{module_name:12s} NOT INSTALLED  ->  pip install {pip_name}")

# %%
print("Hello, Python!")
```

Aussi disponible en notebook Jupyter : python/notebooks/00-getting-started.ipynb

---

Chapitre 01 · 2 sur 17

# Opérateurs

Opérateurs arithmétiques, de comparaison et logiques, ainsi que les tests d’appartenance avec in et index() — les petites briques de base de tout programme.

**▼ Voir le code Python**

○ ○ ○ python/chapters/01-operators.py

```
# %% [markdown]
# # 01 - Operators
# Python can be used as a calculator straight away. Every operator below can be
# typed in a notebook cell, or in the interactive prompt (`python3`).

# %% [markdown]
# ## Arithmetic operators

# %%
print(1 + 1)        # addition
print(5 - 1)        # subtraction
print(3 * 4)        # multiplication
print(7 / 2)        # true division: always a float -> 3.5
print(1035 // 3)    # integer (floor) division -> 345
print(7 % 2)        # modulus (remainder) -> 1
print(2 ** 10)      # exponent -> 1024

# %% [markdown]
# Floor division and modulus follow the sign of the divisor, so
# `a == (a // b) * b + a % b` always holds:
```

```python
# %%
print(-7 // 2, -7 % 3)   # -4  2
print((-7 // 2) * 2 + (-7 % 2))   # -7

# %% [markdown]
# ## Comparison operators
# Each comparison returns `True` or `False`.

# %%
print(1 < 1)
print(1 <= 1)
print(1 > 1)
print(1 >= 1)
print("R" == "r")     # Python is case-sensitive
print(1 != 1)
print(1 < 2 < 3)      # comparisons can be chained

# %% [markdown]
# ## Logical operators
# In plain Python the logical operators are the words `and`, `or`, `not`.

# %%
print(not True)
print(not (True and False))
print(True and True, True and False, False and False)
print(True or True, True or False, False or False)

# %% [markdown]
# On NumPy arrays and pandas columns, use the symbols `&` (and), `|` (or) and
# `~` (not) to combine conditions element by element, with parentheses around
# each condition.

# %%
import numpy as np

scores = np.array([45, 72, 88, 61, 95])
print((scores > 60) & (scores < 90))   # between 60 and 90
print((scores < 50) | (scores > 90))   # very low or very high
print(~(scores > 60))                  # not above 60

# %% [markdown]
# ## Membership and value matching
# `in` tells you whether a value is present. `list.index()` gives its position
# — but remember that Python counts positions **from 0**.

# %%
columns = ["Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width", "Species"]

print("Species" in columns)          # True
print("Weight" not in columns)       # True
print(columns.index("Species"))      # 4  (the 5th element)

# %% [markdown]
# For many values at once on an array, use `np.isin`:

# %%
print(np.isin(["Species", "Weight"], columns))   # [ True False]
```

Aussi disponible en notebook Jupyter : python/notebooks/01-operators.ipynb

---

# Objets, affectation et types de données

Affecter des noms, les règles pour des noms valides, les types de base de Python (int, float, complex, str, bool, bytes, None, catégories) et la façon de les convertir sans risque.

**▼ Voir le code Python**

○ ○ ○ python/chapters/02-objects-assignment-data-types.py

```
# %% [markdown]
# # 02 - Objects, Assignment and Data Types

# %% [markdown]
# ## Assignment
# Python uses a single assignment operator, `=`. A name:
# * may contain letters, digits and underscores, but cannot start with a digit;
# * is case-sensitive (`age` and `Age` are different);
# * cannot be a reserved word such as `for`, `if`, `class`.

# %%
a = 1
b = 2
a1 = a              # a new name for the same value
x, y = 10, 20       # assign several names at once
x, y = y, x         # swap two values
print(a, b, a1, x, y)

count = 5
count += 1          # shorthand for count = count + 1
print(count)

# %%
import keyword

print(keyword.kwlist[:10], "...")

# An invalid name raises a SyntaxError. We catch it here so the notebook goes on.
try:
    compile("v rsion = 4", "<demo>", "exec")      # a space inside a name
except SyntaxError as err:
    print("SyntaxError:", err.msg)

# %% [markdown]
# ## Data types
# Every value has a type. `type()` shows it and `isinstance()` tests it.

# %% [markdown]
# ### int and float (R: integer and numeric)

# %%
n = 55
pi_value = 3.14159
print(type(n), type(pi_value))
print(2 ** 100)          # Python integers can be arbitrarily large
print(0.1 + 0.2)         # floats are approximate: 0.30000000000000004

# %% [markdown]
# ### complex
# The imaginary unit is written `j`: `9 + 3j`.
```

```
# %%
z = 9 + 3j
print(z, type(z), z.real, z.imag)

z1 = a + 1j * b           # build a complex number from two real ones
print(z1, type(z1))

# %% [markdown]
# ### str (R: character)

# %%
text = "I am learning Python"
print(text, type(text), len(text))
print("LeaRning" == "Learning")      # case matters
print(text.upper(), "|", text.split())

# %% [markdown]
# ### bool (R: logical)

# %%
v = True
w = False
print(type(v), not v, v and w)
print(isinstance(True, int), True + True)   # bool is a kind of int: True == 1

# %% [markdown]
# ### bytes (R: raw)

# %%
raw_text = "I am learning Python.".encode("utf-8")
print(raw_text[:12], type(raw_text))
print(raw_text.decode("utf-8"))

# %% [markdown]
# ### None
# `None` represents "no value" (similar to R's `NULL`).

# %%
result = None
print(result, result is None, type(result))

# %% [markdown]
# ### Categories (R: factor)
# For categorical data, pandas provides a `category` type.

# %%
import pandas as pd

gender = pd.Series(["Female", "Male", "Male", "Female"], dtype="category")
print(gender.dtype)
print(gender.cat.categories.tolist())

# %% [markdown]
# ## Conversions
# Convert with the type's own name: `str()`, `int()`, `float()`, `bool()`.

# %%
print(str(55), type(str(55)))            # number -> text
print(int("123") + 1)                    # text -> integer
print(float("3.9"))                      # text -> float
print(int(3.9))                          # float -> int TRUNCATES: 3
print(bool(0), bool(""), bool("a"))      # empty/zero values are False
```

```
# %% [markdown]
# A string that is not a valid number raises an error instead of silently
# becoming a missing value (unlike R). Handle it with `try` / `except`:

# %%
try:
    int("3.9")        # not a valid integer literal
except ValueError as err:
    print("ValueError:", err)

# pandas can instead turn bad values into NaN, like R's NA:
print(pd.to_numeric("abc", errors="coerce"))
print(pd.to_numeric(pd.Series(["10", "x", "30"]), errors="coerce").tolist())
```

Aussi disponible en notebook Jupyter : python/notebooks/02-objects-assignment-data-types.ipynb

---

Chapitre 03 · 4 sur 17

# Séquences, tableaux et matrices

Les tableaux NumPy, des séquences et échantillons aléatoires jusqu'à l'algèbre matricielle — transposée, inverse, systèmes linéaires, valeurs propres — pour finir par une ACP construite à la main sur les données iris.

**▼ Voir le code Python**

○ ○ ○ python/chapters/03-arrays-and-matrices.py

```
# %% [markdown]
# # 03 - Sequences, Arrays and Matrices
# Python's built-in `list` can hold anything, but for numerical work we use
# **NumPy** arrays: fast, one type per array, and arithmetic works on the whole
# array at once (like R vectors and matrices).

# %%
import numpy as np

rng = np.random.default_rng(123)    # reproducible randomness (R: set.seed)

# %% [markdown]
# ## One-dimensional arrays

# %%
mixed = [1, "R", True, None]                  # a list may mix types
print(mixed)

numbers = np.array([1, 4, 8, False, True])    # an array has ONE type: everything -> int
print(numbers, numbers.dtype)

words = np.array([1, "R", True])              # ... or text, if a string is present
print(words, words.dtype)

# %% [markdown]
# ### Sequences

# %%
x = np.linspace(0, 2 * np.pi, 90)             # 90 equally spaced points
print(len(x), round(np.diff(x)[0], 5))        # the common step
print(x[:3], x[-3:])                          # first / last values (indices start at 0)
print(x.min(), x.max(), np.ptp(x))            # range = max - min
```

```
print(np.arange(0, 10, 2))                   # 0, 2, 4, 6, 8 (end is excluded)

# %% [markdown]
# ### Repetition

# %%
print(np.repeat("I learn Python", 2))
print(np.tile([0, 1], 5))              # 0 1 0 1 0 1 0 1 0 1
print(np.repeat([0, 1], 3))            # 0 0 0 1 1 1   (like rep(each = 3))

# %% [markdown]
# ### Random sampling and counting

# %%
y = rng.choice([0, 1], size=200, p=[1/3, 2/3])
values, counts = np.unique(y, return_counts=True)
print(dict(zip(values.tolist(), counts.tolist())))
print(counts / counts.sum())           # proportions
print(np.mean(y == 0))                 # the mean of a boolean array is a proportion

# %% [markdown]
# ### A small survey with typing mistakes

# %%
from collections import Counter

gender = ["Female", "Male", "Male", "Male", "Female", "Female", "Male", "Femaale",
          "Male", "Female", "Malle", "Male", "Female", "Femmale", "Female", "Male"]
print(Counter(gender))                       # the table, typos included

fixes = {"Femaale": "Female", "Femmale": "Female", "Malle": "Male"}
gender = [fixes.get(g, g) for g in gender]   # replace known typos
print(Counter(gender))

# %% [markdown]
# ### Array arithmetic (broadcasting)

# %%
k = 2
v = np.array([1, 0, 3])
print(k + v, k - v, k * v, v / k, k / np.where(v == 0, np.nan, v))
print(np.array([np.pi, 0, 1, 4]) ** 0.5)
print(np.array([1, 2, 3]) + np.array([10, 20, 30]))   # element by element

# %% [markdown]
# ## Matrices (two dimensions)
# NumPy fills a matrix **row by row**; R fills it column by column. Pass
# `order="F"` to `reshape` for R's behaviour.

# %%
A5 = np.arange(1, 26).reshape(5, 5)              # filled by rows
print(A5)
print(np.arange(1, 26).reshape(5, 5, order="F"))  # filled by columns, like R

# %%
M = np.array([[1, 5, 4],
              [0, 2, 2],
              [2, 1, 0]])
print(M.shape)                       # (3, 3)
print(np.zeros((2, 4)))              # a zero matrix

v1, v2, v3 = np.array([1, 0, 2]), np.array([5, 2, 1]), np.array([4, 2, 0])
```

```
print(np.column_stack([v1, v2, v3]))      # R: cbind
print(np.vstack([v1, v2, v3]))            # R: rbind

# %% [markdown]
# ### Matrix operations

# %%
A = M
B = np.array([[2, 3, 0],
              [5, 1, 1],
              [2, 1, 1]])

print(A.T)             # transpose
print(A + B)           # element-wise sum
print(A - B)
print(A * B)           # element-wise product (NOT the matrix product)
print(A @ B)           # matrix product

# %%
A_inv = np.linalg.inv(A)
print(np.round(A_inv, 3))
print(np.round(A @ A_inv))        # the identity matrix

# %% [markdown]
# ### Solving a system of equations
# $$
# \begin{cases} 2x + 2y = 0 \\ x + 3y = 2 \end{cases}
# $$

# %%
A1 = np.array([[2, 2],
               [1, 3]])
b = np.array([0, 2])
print(np.linalg.solve(A1, b))     # x = -1, y = 1

# %% [markdown]
# ### Eigenvalues and eigenvectors
# The eigenvalues $\lambda$ of a square matrix $A$ solve
# $\det(A - \lambda I) = 0$. They are the basis of Principal Component Analysis.

# %%
S = np.array([[2., -1., 0.],
              [-1., 2., -1.],
              [0., -1., 2.]])           # a symmetric matrix
values, vectors = np.linalg.eigh(S)     # eigh: for symmetric matrices
print(np.round(values, 4))
print(np.round(vectors.T @ S @ vectors, 4))   # a diagonal matrix of the eigenvalues

# %% [markdown]
# ## Worked exercise: PCA by hand on the iris data
# 1. Keep the four numeric columns.
# 2. Centre every column.
# 3. Eigen-decompose `X.T @ X`.
# 4. Project onto the first two eigenvectors.

# %%
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

X = load_iris().data                       # 150 flowers x 4 measurements
col_means = X.mean(axis=0)
print(np.round(col_means, 3))
```

```
Xc = X - col_means                          # centred data
cov = Xc.T @ Xc
eigvals, eigvecs = np.linalg.eigh(cov)      # eigenvalues in ascending order
order = np.argsort(eigvals)[::-1]           # sort from largest to smallest
eigvals, eigvecs = eigvals[order], eigvecs[:, order]
print(np.round(eigvals, 3))

projection = Xc @ eigvecs[:, :2]
print(np.round(projection[:3], 3))

# %% [markdown]
# This is exactly what scikit-learn's `PCA` does (the sign of an axis is
# arbitrary, so we compare absolute values):

# %%
scores = PCA(n_components=2).fit_transform(X)
print(np.allclose(np.abs(projection), np.abs(scores)))   # True
```

Aussi disponible en notebook Jupyter : python/notebooks/03-arrays-and-matrices.ipynb

---

Chapitre 04 · 5 sur 17

# Listes, dictionnaires et DataFrames

Listes, tuples, dictionnaires et ensembles, tableaux multidimensionnels, et DataFrames pandas — y compris la transformation d’un âge continu en tranches d’âge lisibles.

**▼ Voir le code Python**

○ ○ ○ python/chapters/04-lists-dicts-dataframes.py

```
# %% [markdown]
# # 04 - Lists, Dictionaries and DataFrames

# %%
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

rng = np.random.default_rng(123)

# %% [markdown]
# ## Multi-dimensional arrays
# NumPy arrays can have any number of dimensions. A classic use is gridded data
# such as maps over time: `array[time, latitude, longitude]`.

# %%
arr = np.arange(1, 25).reshape(2, 3, 4)      # 2 time steps, each a 3 x 4 grid
print(arr.shape)
print(arr[1, 0, 0])      # first row, first column of the 2nd grid
print(arr[0])            # the whole first grid
print(arr.mean(axis=0))  # average over the first dimension

# %% [markdown]
# ## Lists, tuples, dictionaries and sets
# * **list** — ordered and changeable: `[1, 2, 3]`
# * **tuple** — ordered, fixed: `(1, 2)`
# * **dict** — key -> value pairs (like an R named list)
# * **set** — unique values
```

```
# %%
fruits = ["apple", "banana", "cherry"]
fruits.append("mango")
print(fruits, fruits[0], fruits[-1], fruits[1:3], len(fruits))

point = (3, 4)
x, y = point                      # unpacking
print(x, y)

record = {"matrix": np.zeros((2, 2)), "sequence": list(range(1, 6)), "flag": True}
print(record["sequence"], list(record.keys()))
record["text"] = "I am learning Python"      # add a new entry
print(len(record))

print(set([1, 2, 2, 3, 3, 3]))               # {1, 2, 3}

# %% [markdown]
# ## DataFrames
# A pandas `DataFrame` is a table: rows are observations, columns are variables
# and each column can have a different type (R: `data.frame`).

# %% [markdown]
# ### Turning a continuous variable into categories

# %%
n = 120
df = pd.DataFrame({
    "ID": [f"Particip_{i}" for i in range(1, n + 1)],
    "age": rng.choice(np.arange(0, 121), size=n, replace=False),
})
print(df.head())

bins = np.arange(0, 121, 10)
df["age_group"] = pd.cut(df["age"], bins=bins, include_lowest=True)
print(df.head())

# %% [markdown]
# ### Exercise: summarise the age groups
# 1. Counts per age group

# %%
counts = df["age_group"].value_counts().sort_index()
print(counts)

# %% [markdown]
# 2. A bar chart of the groups

# %%
fig, ax = plt.subplots(figsize=(8, 4))
counts.plot.bar(ax=ax, color="purple")
ax.set_title("Distribution of ages")
ax.set_ylabel("participants")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# %% [markdown]
# 3. A histogram of the raw ages

# %%
plt.figure(figsize=(7, 4))
plt.hist(df["age"], bins=12, color="pink", edgecolor="white")
```

```
plt.title("Histogram of ages")
plt.xlabel("age")
plt.show()

# %% [markdown]
# 4. IDs of the participants aged more than 70 up to 80

# %%
ids = df.loc[(df["age"] > 70) & (df["age"] <= 80), "ID"]
print(ids.tolist())

# %% [markdown]
# `groupby` splits the table into one piece per group (R: `split`):

# %%
groups = {key: part for key, part in df.groupby("age_group", observed=True)}
print(list(groups)[:3])
print(groups[pd.Interval(70, 80)]["ID"].tolist())

# %% [markdown]
# ### Creating a DataFrame directly

# %%
df2 = pd.DataFrame({"x": rng.normal(size=10), "y": rng.poisson(2, size=10)})
print(df2.head(3))
print(df2.dtypes)
print(df2.describe().round(2))
```

Aussi disponible en notebook Jupyter : python/notebooks/04-lists-dicts-dataframes.ipynb

---

Chapitre 05 · 6 sur 17

# Nettoyage des données : valeurs manquantes et aberrantes

Repérer et remplir les valeurs NaN, récupérer des nombres piégés dans du texte désordonné, et signaler les valeurs aberrantes avec la méthode de l'IQR avant de les imputer.

**▼ Voir le code Python**

○ ○ ○ python/chapters/05-data-cleaning.py

```
# %% [markdown]
# # 05 - Data Cleaning: Missing Values and Outliers

# %%
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# %% [markdown]
# ## Missing values (`NaN`)
# Missing numbers are represented by `np.nan` ("not a number"; R: `NA`).

# %%
x = np.array([np.nan, 1, 2, np.nan, 3, np.nan, 3.55])

print(np.where(np.isnan(x))[0])        # positions of the missing values
print(np.where(x >= 2)[0])             # NaN compares as False, so it drops out
```

```
print(np.where(~np.isnan(x))[0])       # positions that are NOT missing
print(np.mean(x), np.nanmean(x))       # NaN spreads through a mean; nanmean ignores it

# %% [markdown]
# Replace the missing values with the mean of the others:

# %%
x[np.isnan(x)] = np.nanmean(x)
print(x)

# %% [markdown]
# pandas has dedicated tools: `isna`, `fillna`, `dropna`, `interpolate`.

# %%
s = pd.Series([4.0, np.nan, 7.0, np.nan, 10.0])
print(s.isna().sum(), "missing")
print(s.fillna(s.mean()).tolist())
print(s.dropna().tolist())
print(s.interpolate().tolist())

# %% [markdown]
# ## Missing values created by conversion
# Converting messy text to numbers fails for anything that is not a clean number:

# %%
raw = pd.Series(["2", "1", "2", "7", "$8", "3", "2.5", "9", "2,7"])
y = pd.to_numeric(raw, errors="coerce")        # bad values become NaN
print(y.tolist())
print(raw[y.isna()].tolist())                  # the entries that failed

# %% [markdown]
# Clean the offending characters first, then convert:

# %%
messy = pd.Series(["2", "1", "2", ",7", "$8", "3", "&2.5", "9", "2,7"])
clean = (messy.str.replace(r"[$&]", "", regex=True)      # drop currency / junk symbols
              .str.replace(",", ".", regex=False))       # decimal comma -> dot
print(pd.to_numeric(clean).tolist())

# %% [markdown]
# ## Outlier detection: the IQR method
# A value is an outlier if it lies outside
# $$[Q_1 - 1.5\times IQR,\; Q_3 + 1.5\times IQR]$$
# where $IQR = Q_3 - Q_1$.

# %%
x = np.array([19.103, 19.632, 22.494, 20.113, 10.5, 22.744, 20.737, 17.976,
              18.901, 192.87, 21.959, 0.001, 20.641, 20.177, 19.111, 22.859,
              207.97, 16.853, 21.122, 19.244, 18.291, 19.651, 18.358, 18.834, 19])
x_original = x.copy()

plt.boxplot(x_original)
plt.title("Before handling outliers")
plt.show()

# %%
q1, q2, q3 = np.percentile(x, [25, 50, 75])
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(f"Q1={q1:.3f}  Q3={q3:.3f}  IQR={iqr:.3f}  limits=({lower:.2f}, {upper:.2f})")

is_outlier = (x < lower) | (x > upper)
print(x[is_outlier])                    # the outliers themselves
```

```
# %% [markdown]
# ### Imputing the outliers
# Treat them as missing and fill them with the mean of the remaining values.

# %%
x[is_outlier] = np.nan
x[np.isnan(x)] = np.nanmean(x)

fig, axes = plt.subplots(1, 2, figsize=(8, 4))
axes[0].boxplot(x_original)
axes[0].set_title("Before")
axes[1].boxplot(x)
axes[1].set_title("After")
plt.tight_layout()
plt.show()

# %% [markdown]
# No outliers remain after the replacement (see the right-hand panel).
```

Aussi disponible en notebook Jupyter : python/notebooks/05-data-cleaning.ipynb

---

Chapitre 06 · 7 sur 17

# Graphiques et simulations de lois de probabilité

Graphiques matplotlib en barres, en secteurs, histogrammes et nuages de points, puis simulation de données uniformes, binomiales, normales, exponentielles et de Poisson et ajustement d’un modèle linéaire simple.

**▼ Voir le code Python**

○ ○ ○ python/chapters/06-charts-and-distributions.py

```
# %% [markdown]
# # 06 - Charts and Distribution Simulations
# We use **matplotlib** for plots and NumPy's random generator for simulation.

# %%
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.datasets import load_iris

rng = np.random.default_rng(12092024)

# %% [markdown]
# ## Basic charts
# ### Bar chart and pie chart

# %%
frequencies = {"Female": 35, "Children": 25, "Male": 40}
colours = ["purple", "salmon", "pink"]

fig, axes = plt.subplots(1, 2, figsize=(9, 4))
axes[0].bar(frequencies.keys(), frequencies.values(), color=colours)
axes[0].set_title("Bar chart")
axes[1].pie(frequencies.values(),
            labels=[f"{k} ({v})" for k, v in frequencies.items()], colors=colours)
```

```
axes[1].set_title("Pie chart")
plt.tight_layout()
plt.show()

# %% [markdown]
# ### Histogram and density

# %%
x = rng.integers(1, 121, size=1000)

fig, axes = plt.subplots(1, 2, figsize=(9, 4))
axes[0].hist(x, density=True, color="lightgrey", edgecolor="white")
axes[0].set_title("Histogram (densities)")
grid = np.linspace(x.min(), x.max(), 200)
axes[1].plot(grid, stats.gaussian_kde(x)(grid), color="salmon")
axes[1].set_title("Density of x")
plt.tight_layout()
plt.show()

# %% [markdown]
# ### Scatter plot

# %%
t = np.linspace(0, 2 * np.pi, 50)
plt.scatter(t, np.sin(t), color="blue")
plt.title("sin(x)")
plt.show()

# %% [markdown]
# **Exercise:** `petal length` against `petal width`, coloured by species, with a
# fitted regression line.

# %%
iris = load_iris()
width, length = iris.data[:, 3], iris.data[:, 2]
slope, intercept = np.polyfit(width, length, 1)

for k, name in enumerate(iris.target_names):
    mask = iris.target == k
    plt.scatter(width[mask], length[mask], label=name, s=18)
xs = np.array([width.min(), width.max()])
plt.plot(xs, intercept + slope * xs, "b--", linewidth=2)
plt.xlabel("Petal width")
plt.ylabel("Petal length")
plt.legend()
plt.show()

# %% [markdown]
# ## Simulating distributions
# ### Uniform distribution
# $$f_X(x)=\begin{cases}\frac{1}{b-a} & x\in[a,b]\\ 0 & \text{otherwise}\end{cases}$$

# %%
u = rng.uniform(size=1000)
plt.hist(u, bins=20, density=True, color="turquoise", edgecolor="white")
plt.axvline(u.mean(), color="blue", linestyle="--", label="Average")
plt.title("Histogram of a uniform distribution")
plt.legend()
plt.show()

# %% [markdown]
# A Kolmogorov-Smirnov test: a large p-value is consistent with "looks uniform";
# a Q-Q plot compares the sample with a normal distribution.
```

```
# %%
ks = stats.kstest(u, "uniform")
print(f"KS statistic = {ks.statistic:.3f}, p-value = {ks.pvalue:.3f}")

fig, ax = plt.subplots(figsize=(4.5, 4))
stats.probplot(u, dist="norm", plot=ax)
plt.show()

# %% [markdown]
# ### Binomial distribution
# $$P(X=k)=\binom{n}{k}p^k(1-p)^{n-k}$$

# %%
b = rng.binomial(n=1, p=0.3, size=10000)
values, counts = np.unique(b, return_counts=True)

fig, axes = plt.subplots(1, 2, figsize=(8, 3.5))
axes[0].pie(counts, labels=values)
axes[1].bar(values.astype(str), counts)
plt.tight_layout()
plt.show()

# %% [markdown]
# ### Gaussian (normal) distribution
# $$f_X(x)=\frac{1}{\sigma\sqrt{2\pi}}\exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\}$$

# %%
g = rng.normal(0, 1, size=1000)
plt.hist(g, bins=30, density=True, color="turquoise", edgecolor="white")
grid = np.linspace(-4, 4, 200)
plt.plot(grid, stats.norm.pdf(grid), "r--")
plt.title("Standard normal distribution")
plt.show()
print(round(g.mean(), 3), round(g.std(ddof=1), 3))

# %% [markdown]
# ### A simulated linear relationship
# $$y_i=\alpha+\beta x_i+\varepsilon_i,\qquad \varepsilon_i\sim\mathcal N(0,1)$$

# %%
x = rng.normal(5, 1.5, size=1000)
z = 4 + 8 * x + rng.normal(size=1000)       # the true relationship is known

fit = stats.linregress(x, z)
print(f"intercept = {fit.intercept:.2f}, slope = {fit.slope:.2f}, R^2 = {fit.rvalue**2:.3f}")

# prediction for a new value x = 15
print("prediction at x = 15:", round(fit.intercept + fit.slope * 15, 2))

plt.scatter(x, z, s=6, color="darkorange", label="Actual")
xs = np.array([x.min(), 15])
plt.plot(xs, fit.intercept + fit.slope * xs, "b-", label="Fitted line")
plt.scatter([15], [fit.intercept + fit.slope * 15], color="darkgreen", label="Predicted")
plt.legend()
plt.show()

# %% [markdown]
# ### Exponential and Poisson distributions

# %%
e = rng.exponential(scale=1.0, size=1000)
p = rng.poisson(lam=2.5, size=1000)
```

```
fig, axes = plt.subplots(1, 2, figsize=(9, 3.8))
axes[0].hist(e, bins=30, density=True, color="turquoise", edgecolor="white")
axes[0].set_title("Exponential")
vals, cnt = np.unique(p, return_counts=True)
axes[1].bar(vals, cnt / cnt.sum(), color="turquoise")
axes[1].set_title("Poisson")
plt.tight_layout()
plt.show()
```

Aussi disponible en notebook Jupyter : python/notebooks/06-charts-and-distributions.ipynb

---

Chapitre 07 · 8 sur 17

# Structures de contrôle : conditions et boucles

if/elif/else, boucles for et while, break et continue, travaillés à travers un système de notation, un jeu de dés à séries et la lecture d'un dossier de fichiers en une seule boucle.

**▼ Voir le code Python**

○ ○ ○ python/chapters/07-flow-control.py

```
# %% [markdown]
# # 07 - Flow Control: Conditionals and Loops
# Python marks blocks with **indentation** (4 spaces) instead of braces.

# %%
import numpy as np

rng = np.random.default_rng(123)

# %% [markdown]
# ## if / elif / else

# %%
x = 3
if x < 4:
    print(True)
else:
    print(False)

print(True if x < 4 else False)      # one-line conditional expression

values = np.array([3, 4, 6, 7, 0, 4])
print(np.where(values < 4, True, False))    # vectorised version (R: ifelse)

# %% [markdown]
# ### Grading example
# Categorise each mark: Fail [0, 60), Pass [60, 70), Good Pass [70, 80),
# Very Good Pass [80, 85), Distinction [85, ...).

# %%
marks = np.round(rng.normal(60, 15, size=50), 2)

# Approach 1: loop over the values
categories = []
for m in marks:
    if m < 60:
```

```
        categories.append("Fail")
    elif m < 70:
        categories.append("Pass")
    elif m < 80:
        categories.append("Good Pass")
    elif m < 85:
        categories.append("Very Good Pass")
    else:
        categories.append("Distinction")
print(list(zip(marks[:5], categories[:5])))

# %%
# Approach 2: vectorised with np.select
conditions = [marks < 60, marks < 70, marks < 80, marks < 85]
labels = ["Fail", "Pass", "Good Pass", "Very Good Pass"]
categories2 = np.select(conditions, labels, default="Distinction")
print(list(categories) == categories2.tolist())      # True: same result

# %% [markdown]
# ## for loops

# %%
for i in range(1, 11):          # 1 .. 10  (the end value 11 is excluded)
    if i % 3 == 0:
        print(i, "is a multiple of 3")
    elif i % 2 == 0:
        print(i, "is a multiple of 2")
    else:
        print(i, "is neither a multiple of 2 nor 3")

# %% [markdown]
# ### Exercise 1: odd or even

# %%
for i in range(1, 11):
    print(i, "is even" if i % 2 == 0 else "is odd")

# %% [markdown]
# `enumerate` gives the position and the value; `zip` walks through several
# sequences together:

# %%
names = ["Ada", "Bob", "Chloe"]
ages = [36, 41, 29]
for position, (name, age) in enumerate(zip(names, ages), start=1):
    print(position, name, age)

# %% [markdown]
# ### Exercise 2: import every CSV file in a folder
# We first create a small demo folder, then read all its files in a loop.

# %%
import tempfile
from pathlib import Path

import pandas as pd
from sklearn.datasets import load_iris

folder = Path(tempfile.mkdtemp()) / "demo_csv"
folder.mkdir()
iris = load_iris(as_frame=True).frame
iris.head(3).to_csv(folder / "iris_part.csv", index=False)
iris.tail(3).to_csv(folder / "iris_end.csv", index=False)
```

```
data = {}
for path in sorted(folder.glob("*.csv")):        # every .csv file in the folder
    data[path.name] = pd.read_csv(path)
for name, frame in data.items():
    print(name, frame.shape)

# %% [markdown]
# ## while loops

# %%
i = 0
while i <= 10:
    print(i * 2)
    i += 10

# %% [markdown]
# ### Exercise: a dice streak game
# Roll a die until three 5s appear **in a row**.

# %%
streak, rolls = 0, 0
while streak < 3:
    rolls += 1
    roll = int(rng.integers(1, 7))
    streak = streak + 1 if roll == 5 else 0
print(f"YOU WON! after {rolls} rolls")

# %% [markdown]
# ### `break`, `continue` and `while True`
# `while True` with `break` plays the role of R's `repeat`.

# %%
i = 0
while True:
    i += 1
    if i == 3:
        continue          # skip the rest of this round
    print(i)
    if i >= 5:
        break             # leave the loop

# %% [markdown]
# ### Exercise: an interactive multiplication table
# `input()` waits for the user, so this needs a live session. Put the code in a
# function and call it yourself:

# %%
def multiplication_table():
    while True:
        number = int(input("Enter a number 0-9: "))
        for k in range(1, 11):
            print(f"{number} x {k} = {number * k}")
        if input("Continue? (y/n): ").lower() != "y":
            break

# multiplication_table()      # un-comment to try it in a notebook
```

Aussi disponible en notebook Jupyter : python/notebooks/07-flow-control.ipynb

---

# Compréhensions de liste et vectorisation

Compréhensions de liste, axes NumPy et apply, agg et groupby de pandas — la manière Python d’éviter les boucles explicites, comme la famille apply de R.

**▼ Voir le code Python**

○ ○ ○ python/chapters/08-comprehensions-and-vectorization.py

```python
# %% [markdown]
# # 08 - Comprehensions and Vectorisation
# R has the *apply family* to avoid explicit loops. In Python the same job is
# done by **comprehensions**, `map`/`filter`, NumPy's `axis=` argument and
# pandas' `apply`, `agg` and `groupby`.

# %%
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = df.pop("target").map(dict(enumerate(iris.target_names)))
print(df.head(3))

# %% [markdown]
# ## Comprehensions (R: sapply / lapply)

# %%
squares = [n ** 2 for n in range(1, 11)]
print(squares)

even_squares = [n ** 2 for n in range(1, 11) if n % 2 == 0]    # with a condition
print(even_squares)

lengths = {name: len(name) for name in ["Ada", "Bob", "Chloe"]}   # dict comprehension
print(lengths)

print(list(map(lambda v: v + 1, [1, 2, 3])))
print(list(filter(lambda v: v > 1, [1, 2, 3])))

# %% [markdown]
# ## Along the axes of an array (R: apply on a matrix)
# `axis=0` works down the columns, `axis=1` across the rows.

# %%
A = np.array([[1, 2],
              [3, 4]])
print(A.sum(axis=0), A.sum(axis=1))
print(np.apply_along_axis(lambda row: row.max() - row.min(), 1, A))

# %% [markdown]
# ## DataFrame columns (R: apply / sapply)

# %%
numeric = df.select_dtypes("number")
print(numeric.apply(np.mean))
print(numeric.agg(["min", "max", "std"]).round(2))
print(numeric.apply(lambda col: col.max() - col.min()))      # the range of every column

# %%
```

```
def coef_dispersion(s):
    return s.std() / s.mean()

summary = numeric.apply(lambda s: pd.Series({
    "min": s.min(), "max": s.max(), "std": s.std(),
    "Q1": s.quantile(0.25), "median": s.median(), "Q3": s.quantile(0.75),
    "dispersion": coef_dispersion(s)}))
print(summary.round(3))

# %% [markdown]
# ## Statistics by group (R: tapply)

# %%
print(df.groupby("species")["sepal_length"].mean())
print(df.groupby("species")[["sepal_length", "petal_length"]].agg(["mean", "std"]).round(2))

# %% [markdown]
# ## Quantiles of every column (R: vapply)

# %%
print(numeric.quantile([0, 0.25, 0.5, 0.75, 1]))
```

Aussi disponible en notebook Jupyter : python/notebooks/08-comprehensions-and-vectorization.ipynb

---

Chapitre 09 · 10 sur 17

# Écrire des fonctions en Python

Définir des fonctions avec des valeurs par défaut, plusieurs valeurs de retour, *args et **kwargs, puis deux exercices : remplacer les valeurs manquantes et résoudre une équation du second degré.

**▼ Voir le code Python**

○ ○ ○ python/chapters/09-writing-functions.py

```
# %% [markdown]
# # 09 - Writing Functions in Python

# %% [markdown]
# ## Function syntax
# ```python
# def function_name(arg1, arg2=default):
#     """Docstring: what the function does."""
#     ...
#     return result
# ```

# %%
import numpy as np

rng = np.random.default_rng(1)


def toss_die():
    """Roll one six-sided die."""
    return int(rng.integers(1, 7))


print([toss_die() for _ in range(10)])
```

```
def add_one(x):
    return x + 1


i = 1
i = add_one(i)
print(i)

# %% [markdown]
# ## Default values, keyword arguments and several results

# %%
def describe(values, decimals=2):
    """Return (mean, standard deviation) of a sequence."""
    values = np.asarray(values, dtype=float)
    return round(values.mean(), decimals), round(values.std(ddof=1), decimals)


mean, sd = describe([4, 8, 15, 16, 23, 42])         # a function can return several values
print(mean, sd)
print(describe([4, 8, 15, 16, 23, 42], decimals=0))  # a keyword argument

square = lambda v: v ** 2        # a short anonymous function
print(square(7))

# %%
def total(*numbers, **options):
    """*numbers collects extra positional arguments, **options keyword ones."""
    result = sum(numbers)
    return result * options.get("factor", 1)


print(total(1, 2, 3), total(1, 2, 3, factor=10))

# %% [markdown]
# ## Exercises
# ### 1. Replace missing values with a summary statistic

# %%
def replace_missing(x, fun=np.nanmean):
    """Replace NaN values in x by fun(x), ignoring NaN. Returns a new array."""
    x = np.asarray(x, dtype=float).copy()
    x[np.isnan(x)] = fun(x)
    return x


test = [1, 2, np.nan, 4, np.nan, 6]
print(replace_missing(test))
print(replace_missing(test, fun=np.nanmedian))

# %% [markdown]
# ### 2. Solve a quadratic equation and comment on its discriminant
# For $ax^2+bx+c=0$ the discriminant is $\Delta=b^2-4ac$:
# * $\Delta>0$ — two distinct real roots
# * $\Delta=0$ — one repeated real root
# * $\Delta<0$ — two complex conjugate roots

# %%
import math
```

```python
def solve_quadratic(a, b, c):
    if a == 0:
        raise ValueError("a must be non-zero for a quadratic equation")
    delta = b ** 2 - 4 * a * c
    if delta > 0:
        print("Two distinct real roots")
        return ((-b + math.sqrt(delta)) / (2 * a), (-b - math.sqrt(delta)) / (2 * a))
    if delta == 0:
        print("One repeated real root")
        return (-b / (2 * a),)
    print("Two complex conjugate roots")
    real, imag = -b / (2 * a), math.sqrt(-delta) / (2 * a)
    return (complex(real, imag), complex(real, -imag))


print(solve_quadratic(1, -3, 2))   # delta > 0
print(solve_quadratic(1, 2, 1))    # delta = 0
print(solve_quadratic(1, 0, 1))    # delta < 0

# %% [markdown]
# ## Scope
# A variable created inside a function is local to it.

# %%
counter = 0


def increase():
    counter = 100          # a NEW local variable; the outer one is untouched
    return counter


increase()
print(counter)             # still 0
```

Aussi disponible en notebook Jupyter : python/notebooks/09-writing-functions.ipynb

---

Chapitre 10 · 11 sur 17

# Modules et packages

Importer des modules, un tour de la bibliothèque standard, installer des packages avec pip, et les verbes pandas qui correspondent à ceux de dplyr en R.

**▼ Voir le code Python**

○ ○ ○ python/chapters/10-modules-and-packages.py

```python
# %% [markdown]
# # 10 - Modules and Packages
# A **module** is a file of Python code; a **package** is a collection of
# modules. Python ships with a large *standard library*, and the community
# provides hundreds of thousands more packages on PyPI.

# %% [markdown]
# ## Importing

# %%
import math                       # the whole module
from math import sqrt, pi         # only some names
```

```
import numpy as np               # with a short alias (a universal convention)

print(math.sqrt(16), sqrt(25), round(pi, 4), np.sqrt(36))

# %% [markdown]
# ## A tour of the standard library

# %%
import random
import statistics
from datetime import date
from pathlib import Path

random.seed(1)
print(random.sample(range(1, 50), 6))                # six lottery numbers
print(statistics.mean([2, 4, 4, 4, 5, 5, 7, 9]), statistics.stdev([2, 4, 4, 4, 5, 5, 7, 9]))
print(date(2026, 11, 10).strftime("%A %d %B %Y"))
print(Path.cwd().name)

# %% [markdown]
# ## Finding out what a module offers

# %%
print([name for name in dir(math) if not name.startswith("_")][:8], "...")
print(math.sqrt.__doc__)          # or: help(math.sqrt)

# %% [markdown]
# ## Installing packages
# Packages are installed with **pip**, in a terminal (not in Python code):
# ```bash
# pip install pandas matplotlib scikit-learn
# pip list                      # what is installed
# pip install --upgrade pandas
# ```
# From a notebook cell you can use `%pip install pandas`.
#
# Check from Python whether a package is available, and which version:

# %%
import importlib.metadata as metadata
import importlib.util

for package in ["numpy", "pandas", "not_a_real_package"]:
    if importlib.util.find_spec(package):
        print(f"{package:20s} installed, version {metadata.version(package)}")
    else:
        print(f"{package:20s} NOT installed -> pip install {package}")

# %% [markdown]
# ## pandas verbs (R: the dplyr verbs)
# The data-manipulation vocabulary of R's `dplyr` has a direct counterpart in
# pandas:
#
# | dplyr | pandas |
# |---|---|
# | `select()` | `df[["a", "b"]]`, `df.drop(columns=...)` |
# | `filter()` | `df[condition]`, `df.query("...")` |
# | `arrange()` | `df.sort_values(...)` |
# | `rename()` | `df.rename(columns=...)` |
# | `mutate()` | `df.assign(new=...)` |
# | `group_by() + summarise()` | `df.groupby(...).agg(...)` |

# %%
```

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = df.pop("target").map(dict(enumerate(iris.target_names)))

print(df[["sepal_length", "petal_length"]].head(3))                    # select
print(df.drop(columns="species").columns.tolist())

# %%
print(df[(df["species"] == "setosa") & (df["petal_length"] > 1.4)].head(3))   # filter
print(df.query("species == 'setosa' and petal_length > 1.4").shape)

print(df.sort_values("sepal_length", ascending=False).head(3))                 # arrange
print(df.rename(columns={"petal_length": "PetalLength"}).columns.tolist())     # rename

print(df.assign(ratio=df["petal_length"] / df["petal_width"]).head(3))         # mutate
print(df.groupby("species")["petal_width"].mean().round(3))                    # summarise
```

Aussi disponible en notebook Jupyter : python/notebooks/10-modules-and-packages.ipynb

---

Chapitre 11 · 12 sur 17

# Importer des données

Lire des fichiers CSV, Excel et JSON avec pandas, jeux de données intégrés, et boucler sur un dossier pour réunir plusieurs fichiers en un seul tableau.

**▼ Voir le code Python**

○ ○ ○ python/chapters/11-importing-data.py

```
# %% [markdown]
# # 11 - Importing Data
# > **Note on file paths.** The examples below create small temporary files and
# > read them back, so every cell runs on any computer. To use your own data,
# > replace the temporary path by the path of your file.

# %%
import tempfile
import time
from pathlib import Path

import pandas as pd
from sklearn.datasets import load_iris

workdir = Path(tempfile.mkdtemp())

# %% [markdown]
# ## Built-in data
# scikit-learn ships small example datasets that are ready to use.

# %%
iris = load_iris(as_frame=True)
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = df.pop("target").map(dict(enumerate(iris.target_names)))

df.info()                                   # column names, types, missing values
print(df.describe().round(2))
```

```
print(df.shape)

# %% [markdown]
# Checking the type of every column, timed with the standard `time` module:

# %%
start = time.perf_counter()
dtypes = [str(df[col].dtype) for col in df.columns]
print(f"{time.perf_counter() - start:.6f} seconds")
print(sorted(set(dtypes)))

# %% [markdown]
# ## CSV files (R: read.csv / read_csv)

# %%
csv_path = workdir / "iris.csv"
df.to_csv(csv_path, index=False)            # write
data = pd.read_csv(csv_path)                # read
print(data.head(3))
print(data["species"].dtype)                # text ("object" / "str")

data["species"] = data["species"].astype("category")     # R: stringsAsFactors
print(data["species"].dtype)

# %% [markdown]
# Useful options of `read_csv`: `sep=";"`, `decimal=","`, `na_values=["NA", "-"]`,
# `usecols=[...]`, `nrows=100`, `encoding="latin-1"`.

# %% [markdown]
# ## Excel files (needs the `openpyxl` package: `pip install openpyxl`)

# %%
xlsx_path = workdir / "iris.xlsx"
try:
    df.head(5).to_excel(xlsx_path, index=False)
    print(pd.read_excel(xlsx_path).head(2))
except ImportError:
    print("Install openpyxl first:  pip install openpyxl")

# %% [markdown]
# ## JSON files

# %%
json_path = workdir / "iris.json"
df.head(3).to_json(json_path, orient="records")
print(json_path.read_text()[:120], "...")
print(pd.read_json(json_path).shape)

# %% [markdown]
# ## Exercise: import every CSV file in a folder with a loop

# %%
folder = workdir / "demo_csv"
folder.mkdir()
df.head(3).to_csv(folder / "iris_part.csv", index=False)
df.tail(3).to_csv(folder / "iris_end.csv", index=False)

frames = {}
for path in sorted(folder.glob("*.csv")):
    frames[path.name] = pd.read_csv(path)
    print(path.name, frames[path.name].shape)

combined = pd.concat(frames, names=["file", "row"])     # one table, remembering the file
```

```
print(combined.shape)
print(combined.reset_index(level="file").head(4))
```

Aussi disponible en notebook Jupyter : python/notebooks/11-importing-data.ipynb

---



# Chaînage de méthodes et manipulation de données

Lire les étapes d'une analyse de gauche à droite avec le chaînage de méthodes et .pipe(), remodeler du format large au format long et inversement, fusionner des tableaux et présenter les résultats.

**▼ Voir le code Python**

○ ○ ○ python/chapters/12-method-chaining.py

```
# %% [markdown]
# # 12 - Method Chaining and Data Manipulation
# R's pipe (`%>%` or `|>`) lets you read a sequence of steps from left to right.
# pandas does the same with **method chaining**: every method returns a new
# DataFrame, so you can add `.` after the previous step. Wrapping the chain in
# parentheses lets you put each step on its own line.

# %%
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = df.pop("target").map(dict(enumerate(iris.target_names)))

# %% [markdown]
# ## A first chain

# %%
result = (
    df.groupby("species")["petal_width"]
      .mean()
      .round(3)
      .rename("mean_petal_width")
      .reset_index()
)
print(result)

# %% [markdown]
# Without chaining, the same work needs temporary names (or nested calls that
# read inside-out):

# %%
step1 = df.groupby("species")["petal_width"]
step2 = step1.mean()
print(step2.round(3))

# %% [markdown]
# ## Frequency table of a categorical variable
# `.pipe()` lets you put your own function inside a chain.
```

```
# %%
gender = pd.Series(["Female", "Male", "Male", "Female", "Female", "Male"])

table = (
    gender.value_counts()
          .rename_axis("Gender")
          .reset_index(name="Count")
          .assign(Rel_freq=lambda t: (t["Count"] / t["Count"].sum()).round(2))
)
print(table)

def percent(frame, column):
    return frame.assign(**{column + "_pct": (frame[column] * 100).round(0)})

print(table.pipe(percent, "Rel_freq"))

table.plot.bar(x="Gender", y="Rel_freq", legend=False)
plt.ylabel("Relative frequency")
plt.show()

# %% [markdown]
# ## Reshaping: wide to long and back (R: reshape2 / tidyr)

# %%
stats = df.groupby("species").agg(["mean", "std"])      # columns are (variable, statistic)
print(stats.round(2))

# wide -> long: one row per species / variable / statistic
flat = stats.copy()
flat.columns = [f"{var}|{stat}" for var, stat in flat.columns]       # e.g. "petal_length|mean"
long = flat.reset_index().melt(id_vars="species", value_name="value")
long[["variable", "metric"]] = long["variable"].str.split("|", expand=True)
long = long[["species", "variable", "metric", "value"]]
print(long.head(6).round(2))

# long -> wide again
wide = long.pivot(index=["species", "variable"], columns="metric", values="value").reset_index()
wide["std_error"] = wide["std"] / np.sqrt(len(df))      # standard error of the mean
print(wide.round(3).head(4))

# %%
fig, axes = plt.subplots(1, 4, figsize=(12, 3.2), sharey=False)
for ax, (variable, part) in zip(axes, wide.groupby("variable")):
    ax.bar(part["species"], part["mean"], yerr=part["std_error"], color="forestgreen", alpha=0.6,
           ecolor="orange", capsize=4)
    ax.set_title(variable)
    ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()

# %% [markdown]
# ## Combining tables: merge (R: left_join)

# %%
info = pd.DataFrame({"species": ["setosa", "versicolor", "virginica"],
                     "code": ["SET", "VER", "VIR"]})        # a small look-up table
merged = df.merge(info, on="species", how="left")
print(merged.drop_duplicates("species")[["species", "code"]])

# %% [markdown]
# ## Presenting tables
# * In a notebook the last expression of a cell is displayed as a formatted
```

```
#   table; `df.style` adds colours and formats (needs `jinja2`).
# * `df.to_markdown()` (needs `tabulate`) gives a Markdown table.
# * `df.to_excel("file.xlsx")` and the `python-docx` / `python-pptx` packages
#   create Word and PowerPoint files from Python.

# %%
print(wide[["species", "variable", "mean"]].head(3).round(2).to_markdown(index=False))
```

Aussi disponible en notebook Jupyter : python/notebooks/12-method-chaining.ipynb

---

Chapitre 13 · 14 sur 17

# Visualisation : seaborn et plotly

Graphiques statistiques avec seaborn et graphiques interactifs avec plotly — les équivalents Python de ggplot2 et de plotly en R.

**▼ Voir le code Python**

○ ○ ○ python/chapters/13-visualization.py

```
# %% [markdown]
# # 13 - Visualization: seaborn and plotly
# * **seaborn** builds statistical graphics on top of matplotlib — the Python
#   counterpart of `ggplot2` for everyday plots.
# * **plotly** makes interactive charts (hover, zoom).
#
# Install them with `pip install seaborn plotly`.

# %%
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = df.pop("target").map(dict(enumerate(iris.target_names)))

# %% [markdown]
# ## seaborn
# Describe *which variables go where*; seaborn takes care of the rest.

# %%
sns.set_theme(style="whitegrid")

sns.boxplot(data=df, x="species", y="petal_length", hue="species", legend=False)
plt.title("Petal length by species")
plt.show()

# %% [markdown]
# A grid of small plots, one per species (R: `facet_grid`):

# %%
grid = sns.FacetGrid(df, col="species", height=3)
grid.map_dataframe(sns.histplot, x="petal_length", bins=10)
plt.show()

# %% [markdown]
# Relationships between several variables at once:
```

```python
# %%
sns.pairplot(df, hue="species", height=1.8)
plt.show()

# %% [markdown]
# ## plotly
# `plotly.express` creates interactive figures in one line. In a notebook
# `fig.show()` displays them in place; in a script it opens your browser.

# %%
import plotly.express as px

counts = df["species"].value_counts().rename_axis("species").reset_index(name="n")
fig = px.pie(counts, names="species", values="n", title="Flowers per species")
fig.show()

# %%
fig = px.scatter(df, x="petal_width", y="petal_length", color="species",
                 hover_data=["sepal_length"], title="Hover over the points")
fig.show()

# %% [markdown]
# Save an interactive figure as a stand-alone web page you can share:

# %%
import tempfile
from pathlib import Path

html_path = Path(tempfile.mkdtemp()) / "iris_scatter.html"
fig.write_html(html_path, include_plotlyjs="cdn")      # the page loads plotly.js from the internet
print("saved:", html_path.name, round(html_path.stat().st_size / 1e3), "kB")
```

Aussi disponible en notebook Jupyter : python/notebooks/13-visualization.ipynb

---

Chapitre 14 · 15 sur 17

# Expressions régulières

Détecter, extraire et remplacer des motifs de texte avec le module re et les méthodes de chaînes de pandas.

**▼ Voir le code Python**

○ ○ ○ python/chapters/14-regular-expressions.py

```python
# %% [markdown]
# # 14 - Regular Expressions
# A regular expression (regex) is a pattern used to detect, extract or replace
# text. Python's `re` module provides them, and pandas exposes them through
# `Series.str`.

# %%
import re

text = ["dave@example.com", "call 555-1234", "no match here", "ID: R2024-001"]

# %% [markdown]
# ## Detecting matches
# `re.search` finds a pattern *anywhere* in a string and returns a match object
```

```
# (or `None`).

# %%
print([bool(re.search(r"[0-9]+", t)) for t in text])
print([t for t in text if re.search(r"[0-9]+", t)])        # keep the matching strings

# %% [markdown]
# ## Replacing matches
# `re.sub` replaces **every** match; add `count=1` to replace only the first.

# %%
print([re.sub(r"[0-9]+", "#", t) for t in text])
print([re.sub(r"[0-9]+", "#", t, count=1) for t in text])

# %% [markdown]
# ## Extracting matches
# `re.findall` returns every match; `match.group()` returns the text of one.

# %%
print([re.findall(r"[0-9]+", t) for t in text])
print([m.group() if (m := re.search(r"[0-9]+", t)) else None for t in text])

# %% [markdown]
# ## Capturing parts of a match with groups

# %%
m = re.search(r"(?P<user>[\w.]+)@(?P<domain>[\w.]+)", "write to dave@example.com today")
print(m.group("user"), m.group("domain"), m.span())

# %% [markdown]
# ## A practical pattern: validating an email address
# `re.fullmatch` requires the *whole* string to match.

# %%
email_pattern = re.compile(r"[A-Za-z0-9._-]+@[A-Za-z0-9-]+\.[A-Za-z]{2,}")

for candidate in ["dave@example.com", "bad.email@@x", "no-at-sign.com", "user@nodot", "a.b@c.org"]:
    print(f"{candidate:20s} {bool(email_pattern.fullmatch(candidate))}")

# %% [markdown]
# ## Common building blocks
#
# | Pattern | Matches |
# |---|---|
# | `[0-9]` or `\d` | any digit |
# | `[a-zA-Z]` | any letter |
# | `.` | any single character |
# | `+` | one or more of the previous token |
# | `*` | zero or more of the previous token |
# | `^` / `$` | start / end of the string |
# | `\s` | whitespace |
# | `a\|b` | `a` or `b` |
#
# Write patterns as **raw strings** (`r"..."`) so that backslashes are kept.

# %% [markdown]
# ## Regex in pandas (R: stringr)

# %%
import pandas as pd

s = pd.Series(text)
print(s.str.contains(r"[0-9]+").tolist())
```

```
print(s.str.extract(r"([0-9]+)")[0].tolist())           # first number, NaN if none
print(s.str.replace(r"[0-9]+", "#", regex=True).tolist())
print(s.str.findall(r"[0-9]+").tolist())

# %% [markdown]
# ## Exercise: standardise messy product codes
# Extract only the numeric ID from codes written in different styles.

# %%
codes = ["PRD-2023-001", "prd2023002", "PRD_2023_003", "Prd 2023 004", "PRD-2023-005"]
ids = [re.search(r"(\d{3})\D*$", c).group(1) for c in codes]       # the last three digits
print(ids)
years = [re.search(r"(20\d\d)", c).group(1) for c in codes]
print([f"PRD-{year}-{i}" for year, i in zip(years, ids)])
```

Aussi disponible en notebook Jupyter : python/notebooks/14-regular-expressions.ipynb

---

Chapitre 15 · 16 sur 17

# Introduction à la modélisation : ACP, clustering, KNN et régression

Un premier tour de scikit-learn : ACP et clustering pour trouver une structure sans étiquettes, puis KNN et régression logistique pour prédire une catégorie.

**▼ Voir le code Python**

○ ○ ○ python/chapters/15-intro-to-modeling.py

```
# %% [markdown]
# # 15 - Introduction to Modeling: PCA, Clustering, KNN and Regression
# **scikit-learn** is Python's standard machine-learning library. Every model
# follows the same pattern: create it, `fit` it on data, then `predict` or
# `transform`.

# %%
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris = load_iris()
X, y = iris.data, iris.target
species = iris.target_names

# %% [markdown]
# ## Principal Component Analysis
# Chapter 03 built PCA by hand; `PCA` does the same, plus scaling.

# %%
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)
pca = PCA().fit(X_scaled)
print(np.round(pca.explained_variance_ratio_, 3))      # share of the variance per component

scores = PCA(n_components=2).fit_transform(X_scaled)
for k, name in enumerate(species):
    plt.scatter(scores[y == k, 0], scores[y == k, 1], label=name, s=18)
```

```
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.legend()
plt.title("iris projected onto PC1 / PC2")
plt.show()

# %% [markdown]
# The first two components already explain most of the spread of the data.

# %% [markdown]
# ## Clustering
# ### K-means
# K-means never sees the species; comparing its groups with the real labels is
# only for evaluation.

# %%
from sklearn.cluster import KMeans

km = KMeans(n_clusters=3, n_init=10, random_state=1).fit(X)
print(pd.crosstab(km.labels_, species[y], rownames=["cluster"], colnames=["species"]))

# %% [markdown]
# ### Hierarchical clustering

# %%
from scipy.cluster.hierarchy import dendrogram, fcluster, linkage

subset = X[:20]                                  # a small subset keeps the tree readable
tree = linkage(subset, method="complete")
dendrogram(tree)
plt.title("Hierarchical clustering (first 20 flowers)")
plt.show()
print(fcluster(tree, t=3, criterion="maxclust"))    # cut the tree into 3 groups

# %% [markdown]
# ## K-Nearest Neighbours (KNN)
# A supervised classifier: a flower gets the majority label of its `k` closest
# neighbours in the training data.

# %%
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
knn = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
pred = knn.predict(X_test)
print("accuracy:", round(accuracy_score(y_test, pred), 3))
print(confusion_matrix(y_test, pred))

# %% [markdown]
# ## Simple linear regression
# Petal length explained by petal width (see also chapter 06):

# %%
from sklearn.linear_model import LinearRegression

petal_width, petal_length = X[:, [3]], X[:, 2]
lin = LinearRegression().fit(petal_width, petal_length)
print("intercept:", round(lin.intercept_, 3), " slope:", round(lin.coef_[0], 3))
print("R^2:", round(lin.score(petal_width, petal_length), 3))

# %% [markdown]
```

```
# ## Logistic regression
# For a yes/no outcome we use logistic regression. Here: *virginica* (1) versus
# *versicolor* (0), from the two sepal measurements. scikit-learn applies a
# regularisation penalty by default; a very large `C` switches it off in practice
# and gives the classical logistic regression.

# %%
from sklearn.linear_model import LogisticRegression

mask = y != 0                                   # keep versicolor and virginica
Xs, ys = X[mask][:, :2], (y[mask] == 2).astype(int)

logit = LogisticRegression(C=1e10, max_iter=5000).fit(Xs, ys)
print("coefficients:", np.round(logit.coef_[0], 3), " intercept:", np.round(logit.intercept_, 3))

prob = logit.predict_proba(Xs)[:, 1]
predicted = (prob > 0.5).astype(int)            # turn probabilities into classes at 0.5
print("accuracy:", round(accuracy_score(ys, predicted), 3))
print(confusion_matrix(ys, predicted))

# %% [markdown]
# For p-values and confidence intervals, the `statsmodels` package
# (`pip install statsmodels`) reports a full statistical summary, like R's
# `summary(glm(...))`.
```

Aussi disponible en notebook Jupyter : python/notebooks/15-intro-to-modeling.ipynb

---



# Mathématiques et LaTeX dans les notebooks

Écrire des équations en ligne et en bloc dans les cellules d’un notebook et à l’intérieur des graphiques — une passerelle vers la Partie 3, où LaTeX devient le sujet principal.

**▼ Voir le code Python**

○ ○ ○ python/chapters/16-math-and-latex-in-notebooks.py

```
# %% [markdown]
# # 16 - Math and LaTeX in Python Notebooks
# Jupyter notebooks (and the `# %% [markdown]` cells of these files) render
# LaTeX mathematics, so you can write the equations behind a model next to the
# code that fits it.
#
# ## Inline and display math
# Single dollar signs keep a formula inside the sentence: the variance of
# $X=(X_1,\ldots,X_n)$ is written right in the text.
#
# Double dollar signs put the equation on its own line:
#
# $$
# \operatorname{Var}(X)=\frac{1}{n}\sum_{i=1}^{n}\left(X_i-\bar X\right)^2,
# \qquad \bar X=\frac{1}{n}\sum_{i=1}^{n}X_i
# $$
#
# ## Equations in code output
# `IPython.display.Math` renders a LaTeX string in a notebook cell. (Run as a
# plain script it only prints an object; in a notebook it shows the formula.)
```

```
# %%
try:
    from IPython.display import Math, display

    display(Math(r"f(x) = x^3"))
except ImportError:                      # IPython is installed with Jupyter
    print(r"f(x) = x^3")

# %% [markdown]
# ## Mathematics inside plots
# matplotlib understands a subset of LaTeX ("mathtext") in any text, as long as
# the formula is between dollar signs and the string is a raw string `r"..."`.

# %%
import matplotlib.pyplot as plt
import numpy as np


def cube(x):
    return x ** 3


x = np.linspace(-2, 2, 100)
plt.plot(x, cube(x), label=r"$f(x)=x^3$")
plt.plot(x, np.sin(x), "--", label=r"$g(x)=\sin(x)$")
plt.title(r"Two functions: $f(x)=x^3$ and $g(x)=\sin x$")
plt.xlabel(r"$x$")
plt.ylabel(r"$y=f(x)$")
plt.legend()
plt.show()

print(cube(3))

# %% [markdown]
# ## A fitted model written as an equation
# Use the numbers from a fitted model to build the LaTeX text automatically:

# %%
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression

iris = load_iris()
fit = LinearRegression().fit(iris.data[:, [3]], iris.data[:, 2])

equation = rf"$\widehat{{\mathrm{{length}}}} = {fit.intercept_:.2f} + {fit.coef_[0]:.2f}\,\mathrm{{width}}$"
print(equation)

plt.figure(figsize=(6, 2))
plt.axis("off")
plt.text(0.5, 0.5, equation, ha="center", va="center", fontsize=18)
plt.show()

# %% [markdown]
# ## From notebook to PDF
# `jupyter nbconvert --to pdf notebook.ipynb` produces a PDF; it needs a LaTeX
# distribution such as TeX Live or MiKTeX (see Part 3 of the course). To use
# full LaTeX for the text inside matplotlib figures, set
# `plt.rcParams["text.usetex"] = True` (also requires LaTeX).
```

Aussi disponible en notebook Jupyter : python/notebooks/16-math-and-latex-in-notebooks.ipynb

# LaTeX

22 leçons · Enseignement : Dr. Lema Logamou Seknewna

22 sections sur la composition de documents en écrivant plutôt qu’en cliquant.

---

Section 01 · 1 sur 22

## Introduction

*Composer des documents en écrivant plutôt qu’en cliquant. 22 diapositives, du fichier vide à votre premier document correctement structuré.*

Tutoriel complet d’Overleaf (en anglais)Essayer en ligne (Overleaf)

---

Section 02 · 2 sur 22 Notions

## Qu’est-ce que LaTeX ?

La plupart des traitements de texte sont **WYSIWYG** — « ce que vous voyez est ce que vous obtenez ». Vous cliquez, faites glisser et mettez en forme la page directement, et l’outil vous montre le résultat au fur et à mesure.

LaTeX fonctionne à l’inverse. Vous écrivez un fichier texte brut rempli de commandes qui décrivent *ce que sont les choses* — une section, un tableau, une équation — et non leur apparence au pixel près. Un programme appelé **moteur TeX** lit ensuite ce fichier et le compile en un PDF final.

En résumé : vous vous concentrez sur le contenu et la structure, et LaTeX s’occupe de la mise en forme visuelle, de façon cohérente, à chaque fois.

---

Section 03 · 3 sur 22 Notions

## Pourquoi apprendre LaTeX ?

- Gère les formules complexes, les notations techniques et les grands tableaux bien plus élégamment qu’un traitement de texte.
- Numérote automatiquement les figures, les équations, les citations et les entrées de la table des matières — renumérotez librement sans rien casser.
- Sépare le contenu du style : changez l’apparence d’un document sans toucher à un seul mot de ce qu’il dit.
- Extensible grâce à des milliers de packages gratuits écrits par la communauté.

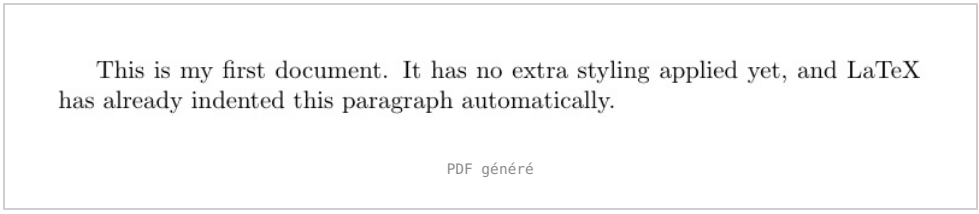
Ce n’est pas l’outil idéal pour tout — les mises en page libres de type magazine restent plus faciles dans un outil de conception graphique. Mais pour tout ce qui est technique, universitaire ou riche en références, l’effort d’apprentissage est vite rentabilisé.

Section 04 · 4 sur 22 Votre premier document

# Documentclass et corps du document

main.tex

```
\documentclass{article}
\begin{document}
This is my first document. It has no extra
styling applied yet, and LaTeX has already
indented this paragraph automatically.
\end{document}
```




`\documentclass` définit le type de document. Tout ce qui se trouve entre `\begin{document}` et `\end{document}` constitue le corps — la partie qui est réellement composée.

Section 05 · 5 sur 22 Structure

# Le préambule

Tout ce qui se trouve *avant* `\begin{document}` s'appelle le **préambule** — la section de configuration du document.

préambule

```
\documentclass[12pt, a4paper]{article}
\usepackage{amsmath}
```

- Les options entre `[...]` configurent la classe — ici, une police de 12 points sur papier A4.
- `\usepackage{...}` charge des fonctionnalités supplémentaires — nous y reviendrons dans quelques diapositives.

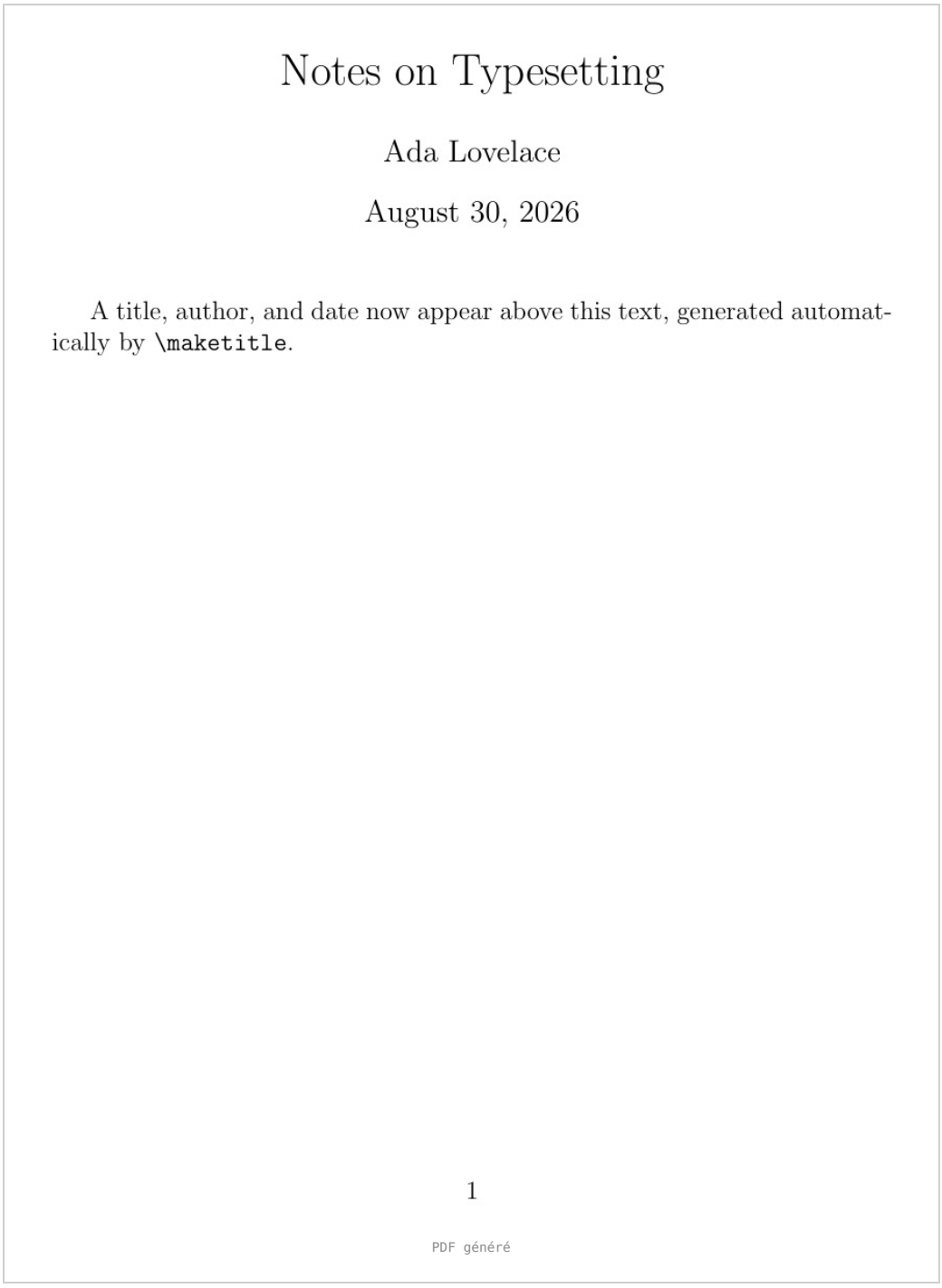
Section 06 · 6 sur 22 Structure

# Titre, auteur et date

main.tex

```
\documentclass[12pt, letterpaper]{article}
\title{Notes on Typesetting}
\author{Ada Lovelace}
\date{\today}
\begin{document}
\maketitle
A title, author, and date now appear above
this text, generated automatically by
\maketitle.
```

```
\end{document}
```




`\date{\today}` insère automatiquement la date réelle de compilation, à chaque recompilation du document.

Section 07 · 7 sur 22 Structure

# Commentaires

main.tex

```
% Everything after a percent sign is ignored.
% Use comments for to-do notes, explanations,
% or to disable a line while debugging.
\documentclass{article}
```

Un `%` en début de ligne — ou n'importe où en milieu de ligne — commente tout ce qui suit. Rien de ce qui suit un `%` n'est composé.

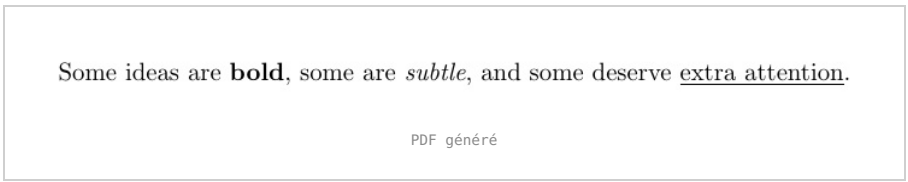
---

Section 08 · 8 sur 22 Mise en forme

# Gras, italique et souligné

main.tex

```
Some ideas are \textbf{bold}, some
are \textit{subtle}, and some
deserve \underline{extra attention}.
```




Il existe aussi `\emph{...}`, qui s'adapte au contexte : italique dans un texte droit, mais retour au droit si vous êtes déjà dans un bloc en italique.

Section 09 · 9 sur 22 Mise en forme

# Listes

main.tex

```
\begin{itemize}
  \item First point
  \item Second point
\end{itemize}

\begin{enumerate}
  \item Step one
  \item Step two
\end{enumerate}
```




`itemize` donne des puces, `enumerate` donne une numérotation automatique — même syntaxe `\item` dans les deux cas.

Section 10 · 10 sur 22 Médias

# Ajouter des images

L'importation d'une image nécessite le package `graphicx`, chargé dans le préambule :

main.tex

```
\usepackage{graphicx}
\graphicspath{{figures/}}

\includegraphics[width=0.75\textwidth]{diagram}
```

- `\graphicspath` indique à LaTeX dans quel dossier chercher.
- Donnez le nom du fichier sans son extension — LaTeX cherche lui-même parmi les formats pris en charge.
- `width=0.75\textwidth` redimensionne l'image par rapport à la largeur du texte, et non à une taille absolue.

Section 11 · 11 sur 22 Médias

# Légendes, étiquettes et références croisées

main.tex

```
\begin{figure}[h]
  \centering
  \includegraphics[width=0.6\textwidth]{diagram}
  \caption{A labeled diagram.}
  \label{fig:diagram}
\end{figure}

See Figure~\ref{fig:diagram} on
page~\pageref{fig:diagram}.
```

`\label` + `\ref` sont tout le secret : LaTeX suit lui-même le numéro réel de la figure. Réorganisez ou insérez des figures librement — chaque référence se met à jour à la compilation suivante.

Section 12 · 12 sur 22 Mathématiques

# Mode mathématique : en ligne ou en bloc

main.tex

```
Inline: Einstein's relation
$E=mc^2$ sits right in a sentence.

Display:
\[ \int_0^1 x^2\,dx = \frac{1}{3} \]

Greek and subscripts:
$\alpha$, $\beta$, $\Omega$, $x_i^2$
```

Inline: Einstein's relation $E = mc^2$ sits right in a sentence.

Display:

$$\int_0^1 x^2\,dx = \frac{1}{3}$$

Greek letters and subscripts: $\alpha$, $\beta$, $\Omega$, $x_i^2$

PDF généré

Un simple `$...$` garde la formule dans le fil du texte. `\[...\]` centre une équation sur sa propre ligne — préférez-le à l'ancien `$$...$$`, que le LaTeX moderne déconseille.

Section 13 · 13 sur 22 Mathématiques

# Éléments mathématiques courants

| Je veux | J'écris |
|---|---|
| Fraction | `\frac{a}{b}` |
| Indice / exposant | `a_b` / `a^b` |
| Racine carrée | `\sqrt{x}` |
| Intégrale | `\int_0^1 ... \, dx` |
| Sommation | `\sum_{i=1}^n` |
| Lettres grecques | `\alpha \beta \Omega` |
| Fonctions nommées | `\sin \cos \log` |

Tout cela fonctionne aussi bien dans `$...$` que dans `\[...\]`. Les fonctions nommées comme `\sin` comptent — un simple `sin(x)` s'affiche en italique, comme si c'étaient trois variables multipliées.

Section 14 · 14 sur 22 Structure du document

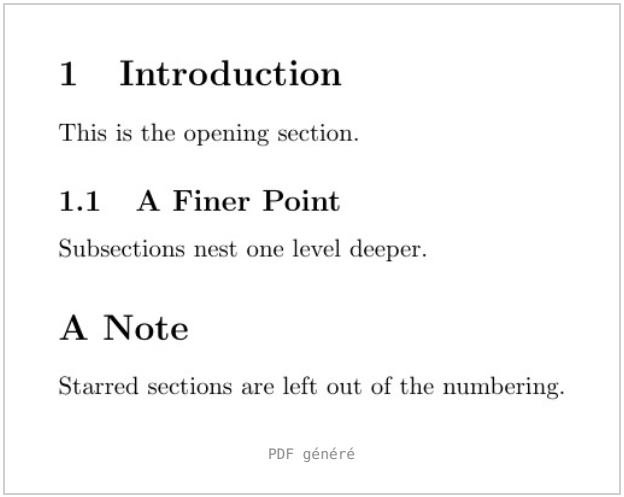
# Sections et chapitres

main.tex

```
\section{Introduction}
This is the opening section.

\subsection{A Finer Point}
Subsections nest one level deeper.

\section*{A Note}
Starred sections skip numbering.
```




`\chapter` et `\part` n'existent que dans les classes `report` et `book` — `article` s'arrête à `\section`. La numérotation est automatique, sauf si vous utilisez la forme étoilée (`*`).

Section 15 · 15 sur 22 Structure du document

# Paragraphes et sauts de ligne

main.tex

```
First paragraph.

Second paragraph starts after a blank line.

Third paragraph continues on\\
a manual new line, same paragraph.
```

- Une ligne vide commence un nouveau paragraphe — LaTeX l'indente automatiquement.
- `\\` (ou `\newline`) coupe la ligne sans commencer un nouveau paragraphe.

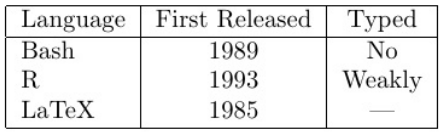
Évitez d'empiler des `\\\\` pour simuler un espacement supplémentaire entre paragraphes — cela contrarie les règles d'espacement propres à LaTeX. Utilisez plutôt le package `parskip` si vous voulez des espaces visibles.

Section 16 · 16 sur 22 Tableaux

# Construire un tableau

main.tex

```
\begin{tabular}{|l|c|c|}
\hline
Language & First Released & Typed \\
\hline
Bash & 1989 & No \\
R & 1993 & Weakly \\
LaTeX & 1985 & --- \\
\hline
\end{tabular}
```



| Language | First Released | Typed |
|---|---|---|
| Bash | 1989 | No |
| R | 1993 | Weakly |
| LaTeX | 1985 | — |

PDF généré

`{|l|c|c|}` déclare trois colonnes (alignées à gauche, au centre, au centre) avec des filets verticaux entre elles. `&` sépare les cellules, `\\` termine une ligne, et `\hline` trace un filet horizontal.

Section 17 · 17 sur 22 Structure du document

# Table des matières

main.tex

```
\tableofcontents

\section{Background}
...
\section{Findings}
...
```

Une seule commande construit et numérote toute la table des matières à partir des commandes `\section` / `\subsection` déjà présentes dans votre document. Compilez deux fois — la première passe collecte les numéros de sections et de pages, la seconde les écrit dans la page de la table des matières.

Section 18 · 18 sur 22 Packages

# Charger des packages et leurs options

main.tex

```
\usepackage{geometry}
\usepackage[margin=1in]{geometry}
```

Les options entre `[...]` configurent le comportement d'un package — ici, des marges de 1 pouce sur toutes les pages. Sans options, un package se charge avec ses valeurs par défaut.

Si LaTeX ne trouve pas le fichier `.sty` du package, il s'arrête avec une erreur claire « fichier introuvable » plutôt que d'échouer silencieusement.

Section 19 · 19 sur 22 Packages

# Trouver des packages : CTAN et TeX Live

**CTAN** (Comprehensive TeX Archive Network) héberge des milliers de packages gratuits fournis par la communauté, de la notation chimique aux diagrammes d'échecs.

**TeX Live** regroupe un instantané important et testé du CTAN en une distribution installable, mise à jour chaque année. Overleaf, et la plupart des installations LaTeX de bureau, reposent sur TeX Live — un package peut donc exister sur le CTAN sans encore être disponible chez vous s'il est très récent ou pose des problèmes de licence ou de plateforme.

Parcourir le CTANÀ propos de TeX Live

Section 20 · 20 sur 22 Référence

# Aide-mémoire

| Commande | Rôle |
|---|---|
| `\documentclass{article}` | Définit le type de document |
| `\usepackage{name}` | Charge des fonctionnalités supplémentaires |
| `\section{...}` | Titre numéroté |
| `\textbf{} \textit{}` | Gras / italique |
| `itemize` / `enumerate` | Liste à puces / numérotée |
| `$...$` / `\[...\]` | Formule en ligne / en bloc |
| `tabular` | Environnement de tableau |
| `\label` / `\ref` | Référence croisée à n'importe quoi |
| `\tableofcontents` | Table des matières générée automatiquement |

Section 21 · 21 sur 22 Continuer

# Pour aller plus loin

Cette série couvre les mêmes notions que l'introduction gratuite d'Overleaf, condensées et restructurées sous forme de diapositives — leur version est plus détaillée sur chaque sujet et mérite d'être lue en entier.

Overleaf : Learn LaTeX in 30 minutes (en anglais)Accueil de la documentation Overleaf

Au-delà des bases : la gestion de bibliographie (`biblatex` / `bibtex`), la classe `Beamer` pour créer de vraies présentations directement en LaTeX, et `TikZ` pour dessiner des diagrammes dans votre document.

Section 22 · 22 sur 22

# Et ensuite ?

*La partie 4 le met en pratique : préparation à l'épidémiologie mathématique en R et en Python.*



# Préparation à l’épidémiologie mathématique

8 leçons · Enseignement : Dr. Lema Logamou Seknewna

8 courts chapitres qui préparent les étudiants à un cours d’épidémiologie mathématique, chacun rédigé deux fois, en R et en Python, avec les mêmes paramètres afin de pouvoir comparer les résultats ligne par ligne. Ils n’utilisent que des exemples de base : taux de croissance, modèles SIR et SEIR, R0 et immunité collective, vaccination, flambées stochastiques et ajustement d’un modèle à des données. Installation : `install.packages("deSolve")` pour R, et `pip install numpy scipy matplotlib` pour Python.

---

Chapitre 00 · 1 sur 8

## Boîte à outils : R et Python côte à côte

Les quelques outils dont dépend chaque chapitre suivant (fonctions, vecteurs, boucles et graphiques), présentés à travers le taux de reproduction de base R0 = β/γ, avec un aide-mémoire R–Python pour passer d’un langage à l’autre.

**▼ Voir le R Markdown**

○ ○ ○ mathepi/r/00-toolkit.Rmd

````
---
title: "MathEpi 00 - Toolkit: R and Python Side by Side"
subtitle: "Mathematical Epidemiology Prep (R)"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE, fig.height = 4, fig.width = 7)
```

Models in epidemiology are built from a handful of numbers (rates, populations,
time steps) and a handful of operations (functions, loops, plots). This
chapter shows the basic R tools you will use, and, in the Python version of
this chapter, the same tools in Python.

# One idea to practice with: the basic reproduction number

The basic reproduction number $R_0$ is the average number of people one
infectious person infects in a fully susceptible population. In the simplest
model

$$R_0 = \frac{\beta}{\gamma}$$
````

````
where $\beta$ is the transmission rate (per day) and $\gamma$ is the recovery
rate (per day), so $1/\gamma$ is the average infectious period.

```{r}
r0 <- function(beta, gamma) beta / gamma

r0(beta = 0.5, gamma = 0.2)   # 2.5: one case infects 2.5 others on average
```

# Vectors and loops

```{r}
betas <- c(0.2, 0.4, 0.6, 0.8)
gamma <- 0.25

for (b in betas) {
  cat("beta =", b, " -> R0 =", r0(b, gamma), "\n")
}

r0(betas, gamma)              # functions work on whole vectors at once
```

# A data frame of scenarios

```{r}
scenarios <- data.frame(beta = betas, gamma = gamma)
scenarios$R0 <- r0(scenarios$beta, scenarios$gamma)
scenarios$can_spread <- scenarios$R0 > 1   # an outbreak needs R0 > 1
scenarios
```

# A first plot

```{r}
plot(betas, r0(betas, gamma), type = "b", pch = 16,
     xlab = "Transmission rate (beta)", ylab = "R0",
     main = "R0 grows with the transmission rate")
abline(h = 1, lty = 2)          # the threshold R0 = 1
```

# R and Python cheat sheet

| Task | R | Python (numpy) |
|---|---|---|
| A vector / array | `c(1, 2, 3)` | `np.array([1, 2, 3])` |
| Sequence | `seq(0, 1, by = 0.1)` | `np.arange(0, 1.01, 0.1)` or `np.linspace(0, 1, 11)` |
| Length | `length(x)` | `len(x)` |
| First element | `x[1]` | `x[0]` (Python counts from 0) |
| Loop | `for (i in 1:3) { ... }` | `for i in range(3): ...` |
| Function | `f <- function(x) x^2` | `def f(x): return x**2` |
| Power | `x^2` | `x**2` |
| Plot | `plot(x, y)` | `plt.plot(x, y)` |
````

**▼ Voir le script Python**

○ ○ ○ mathepi/python/00-toolkit.py

```
"""
MathEpi 00 - Toolkit: R and Python side by side
```

```
Run with:  python 00-toolkit.py
Needs:     numpy, matplotlib   (pip install numpy matplotlib)

The basic reproduction number R0 is the average number of people one
infectious person infects in a fully susceptible population.
In the simplest model:  R0 = beta / gamma
  beta  = transmission rate (per day)
  gamma = recovery rate (per day), so 1/gamma is the infectious period
"""
import numpy as np
import matplotlib.pyplot as plt


# A function (R: r0 <- function(beta, gamma) beta / gamma)
def r0(beta, gamma):
    return beta / gamma


print("R0 =", r0(beta=0.5, gamma=0.2))   # 2.5

# Arrays and loops (R: betas <- c(0.2, 0.4, 0.6, 0.8))
betas = np.array([0.2, 0.4, 0.6, 0.8])
gamma = 0.25

for b in betas:
    print(f"beta = {b} -> R0 = {r0(b, gamma):.1f}")

print(r0(betas, gamma))   # numpy functions work on whole arrays at once

# A small table of scenarios (R: a data frame)
print("\n beta   gamma   R0    can_spread")
for b in betas:
    R0 = r0(b, gamma)
    print(f" {b:.1f}    {gamma:.2f}   {R0:.1f}   {R0 > 1}")   # an outbreak needs R0 > 1

# A first plot
plt.plot(betas, r0(betas, gamma), "o-")
plt.axhline(1, linestyle="--", color="grey")    # the threshold R0 = 1
plt.xlabel("Transmission rate (beta)")
plt.ylabel("R0")
plt.title("R0 grows with the transmission rate")
plt.show()

# ---------------------------------------------------------------
# R vs Python cheat sheet
#   Task            R                       Python (numpy)
#   array           c(1, 2, 3)              np.array([1, 2, 3])
#   sequence        seq(0, 1, by = 0.1)     np.linspace(0, 1, 11)
#   length          length(x)               len(x)
#   first element   x[1]                    x[0]   <- Python counts from 0!
#   loop            for (i in 1:3) { ... }  for i in range(3): ...
#   function        f <- function(x) x^2    def f(x): return x**2
#   power           x^2                     x**2
#   plot            plot(x, y)              plt.plot(x, y)
# ---------------------------------------------------------------
```

---



# Croissance exponentielle et temps de doublement

Au début d'une épidémie, les cas croissent de façon exponentielle. Simulez des comptages quotidiens bruités, estimez le taux de croissance par un ajustement log-linéaire, et convertissez-le en temps de doublement.

**▼ Voir le R Markdown**

○ ○ ○ mathepi/r/01-exponential-growth.Rmd

````
---
title: "MathEpi 01 - Exponential Growth and Doubling Time"
subtitle: "Mathematical Epidemiology Prep (R)"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE, fig.height = 4, fig.width = 7)
```

At the start of an outbreak, almost everyone is susceptible, so cases grow
roughly exponentially:

$$I(t) \approx I_0\, e^{rt}$$

The growth rate $r$ can be estimated from data, and from it the
**doubling time** $T_d = \ln(2)/r$.

# Simulate daily case counts

We pretend the true growth rate is $r = 0.15$ per day, starting from 5 cases,
and add Poisson noise because real counts are never perfectly smooth.

```{r}
set.seed(1)
r_true <- 0.15
day <- 0:29
cases <- rpois(length(day), lambda = 5 * exp(r_true * day))
data <- data.frame(day, cases)
head(data)
```

# Estimate the growth rate

Taking logs turns the exponential curve into a straight line:
$\log I(t) = \log I_0 + r\,t$. So a linear regression of `log(cases)` on `day`
estimates $r$ as the slope.

```{r}
fit <- lm(log(cases) ~ day, data = data)
r_hat <- unname(coef(fit)["day"])
r_hat                       # close to the true value 0.15
log(2) / r_hat              # estimated doubling time in days
log(2) / r_true             # true doubling time
```

# Plot data and fit on a log scale
````

```{r}
plot(day, cases, log = "y", pch = 16,
     xlab = "Day", ylab = "Daily cases (log scale)")
lines(day, exp(predict(fit)), col = "red", lwd = 2)
```

On a log scale, exponential growth is a straight line. Real outbreaks leave
this straight line once susceptibles are depleted or control measures start.

**▼ Voir le script Python**

○ ○ ○ mathepi/python/01-exponential-growth.py

```
"""
MathEpi 01 - Exponential growth and doubling time

At the start of an outbreak almost everyone is susceptible, so cases grow
roughly exponentially:   I(t) ~ I0 * exp(r * t)
The growth rate r can be estimated from data, and the doubling time is
    T_d = ln(2) / r
"""
import numpy as np
import matplotlib.pyplot as plt

# Simulate daily case counts: true r = 0.15 per day, 5 cases on day 0,
# plus Poisson noise (real counts are never perfectly smooth).
rng = np.random.default_rng(1)          # (R: set.seed(1))
r_true = 0.15
day = np.arange(30)                      # days 0..29
cases = rng.poisson(5 * np.exp(r_true * day))
print("first days:", cases[:6])

# Taking logs turns the exponential into a straight line:
#     log I(t) = log I0 + r * t
# so a straight-line fit of log(cases) against day gives r as the slope.
slope, intercept = np.polyfit(day, np.log(cases), 1)
r_hat = slope
print(f"estimated r      = {r_hat:.3f}   (true {r_true})")
print(f"doubling time    = {np.log(2) / r_hat:.2f} days")
print(f"true doubling    = {np.log(2) / r_true:.2f} days")

# Plot data and fit on a log scale (a straight line = exponential growth)
plt.semilogy(day, cases, "o", label="Observed")
plt.semilogy(day, np.exp(intercept + slope * day), "r-", label="Fitted")
plt.xlabel("Day")
plt.ylabel("Daily cases (log scale)")
plt.legend()
plt.show()
```

---

Chapitre 02 · 3 sur 8

# Le modèle SIR

Le modèle classique à trois compartiments : Susceptibles, Infectieux, Rétablis. Résolvez ses équations différentielles numériquement avec deSolve en R et solve_ivp en Python, puis lisez le pic et le nombre total de personnes infectées.

**▼ Voir le R Markdown**

mathepi/r/02-sir-model.Rmd

````
---
title: "MathEpi 02 - The SIR Model"
subtitle: "Mathematical Epidemiology Prep (R)"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE, fig.height = 4, fig.width = 7)
```

The SIR model splits a population of size $N$ into three compartments:
**S**usceptible, **I**nfectious, and **R**ecovered (or removed).

$$
\frac{dS}{dt} = -\beta \frac{S I}{N}, \qquad
\frac{dI}{dt} = \beta \frac{S I}{N} - \gamma I, \qquad
\frac{dR}{dt} = \gamma I
$$

Susceptibles become infectious at rate $\beta S I / N$; infectious people
recover at rate $\gamma$. The three equations add up to zero, so the total
population $N$ is constant.

# Solve it numerically with `deSolve`

```{r}
library(deSolve)

sir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

N     <- 1000
init  <- c(S = N - 1, I = 1, R = 0)
parms <- c(beta = 0.5, gamma = 0.2, N = N)
times <- seq(0, 120, by = 1)

out <- as.data.frame(ode(y = init, times = times, func = sir, parms = parms))
head(out)
```

# Plot the epidemic

```{r}
matplot(out$time, out[, c("S", "I", "R")], type = "l", lty = 1, lwd = 2,
        col = c("blue", "red", "darkgreen"),
        xlab = "Day", ylab = "People", main = "SIR epidemic")
````
````

```
legend("right", c("S", "I", "R"), col = c("blue", "red", "darkgreen"), lty = 1, lwd = 2)
```

# Read off the key numbers

```{r}
peak_I   <- max(out$I)
peak_day <- out$time[which.max(out$I)]
c(peak_infectious = round(peak_I), peak_day = peak_day,
  total_ever_infected = round(max(out$R)))
```

The epidemic rises, peaks when $S/N = 1/R_0$, and then burns out, leaving
some people who were never infected.

**▼ Voir le script Python**

○ ○ ○ mathepi/python/02-sir-model.py

```
"""
MathEpi 02 - The SIR model

Susceptible -> Infectious -> Recovered, in a population of size N:

    dS/dt = -beta * S * I / N
    dI/dt =  beta * S * I / N - gamma * I
    dR/dt =  gamma * I

Needs: numpy, scipy, matplotlib
"""
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt


def sir(t, y, beta, gamma, N):
    S, I, R = y
    dS = -beta * S * I / N
    dI = beta * S * I / N - gamma * I
    dR = gamma * I
    return [dS, dI, dR]


N = 1000
beta, gamma = 0.5, 0.2
y0 = [N - 1, 1, 0]                      # S, I, R at time 0
days = np.arange(0, 121)                # 0, 1, ..., 120

# rtol sets how accurately the solver works; a tight value gives the same
# numbers as R's deSolve
sol = solve_ivp(sir, (0, 120), y0, args=(beta, gamma, N), rtol=1e-8, t_eval=days)
S, I, R = sol.y

plt.plot(sol.t, S, "b", label="S")
plt.plot(sol.t, I, "r", label="I")
plt.plot(sol.t, R, "g", label="R")
plt.xlabel("Day")
plt.ylabel("People")
plt.title("SIR epidemic")
plt.legend()
plt.show()
```

```
print(f"peak infectious     = {I.max():.0f}")
print(f"peak day            = {sol.t[I.argmax()]:.0f}")
print(f"total ever infected = {R[-1]:.0f}")
```

---

Chapitre 03 · 4 sur 8

# R0, immunité collective et taille finale

Trois nombres qui résument une épidémie : R0, le seuil d'immunité collective 1 − 1/R0, et la taille finale de l'épidémie, obtenue en résolvant numériquement $z = 1 - e^{-R0 \cdot z}$ puis vérifiée par une simulation.

**▼ Voir le R Markdown**

○ ○ ○ mathepi/r/03-r0-herd-immunity.Rmd

````
---
title: "MathEpi 03 - R0, Herd Immunity and Final Size"
subtitle: "Mathematical Epidemiology Prep (R)"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE, fig.height = 4, fig.width = 7)
```

Three quantities summarise most of what the SIR model says about an outbreak.

* **Basic reproduction number:** $R_0 = \beta/\gamma$. An outbreak can take
  off only if $R_0 > 1$.
* **Herd immunity threshold:** the fraction of the population that must be
  immune so that $R_0 \times (\text{fraction susceptible}) \le 1$, i.e.
  $1 - 1/R_0$.
* **Final size:** the fraction $z$ of the population ever infected, which
  solves $z = 1 - e^{-R_0 z}$ (when almost everyone starts susceptible).

```{r}
beta <- 0.5
gamma <- 0.2
R0 <- beta / gamma
herd_immunity <- 1 - 1 / R0
c(R0 = R0, herd_immunity_threshold = herd_immunity)
```

# Solving for the final size

The final-size equation has no closed-form solution, so we find its root
numerically with `uniroot()`.

```{r}
final_size <- function(R0) {
  if (R0 <= 1) return(0)                      # no major outbreak
````
````

```
  uniroot(function(z) z - (1 - exp(-R0 * z)),
          lower = 1e-6, upper = 1)$root
}

final_size(R0)
```

# Check against the simulated epidemic

```{r}
library(deSolve)
sir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}
N <- 1000
out <- as.data.frame(ode(c(S = N - 1, I = 1, R = 0), seq(0, 300, by = 1),
                         sir, c(beta = beta, gamma = gamma, N = N)))
tail(out$R, 1) / N            # fraction ever infected in the simulation
```

The simulation and the equation agree.

# How the final size depends on R0

```{r}
R0_values <- c(1.2, 1.5, 2, 2.5, 3, 4)
table_out <- data.frame(
  R0 = R0_values,
  herd_immunity = round(1 - 1 / R0_values, 3),
  final_size = round(sapply(R0_values, final_size), 3)
)
table_out
```

```{r}
curve_R0 <- seq(1.05, 5, by = 0.05)
plot(curve_R0, sapply(curve_R0, final_size), type = "l", lwd = 2,
     xlab = "R0", ylab = "Fraction ever infected",
     main = "Final epidemic size")
lines(curve_R0, 1 - 1 / curve_R0, lty = 2)
legend("bottomright", c("Final size", "Herd immunity threshold"), lty = c(1, 2))
```

Notice that the final size is always larger than the herd immunity threshold:
the epidemic overshoots because infections continue while $I$ is still large.

**▼ Voir le script Python**

○ ○ ○ mathepi/python/03-r0-herd-immunity.py

```
"""
MathEpi 03 - R0, herd immunity and final size

  R0                = beta / gamma         (an outbreak needs R0 > 1)
  herd immunity     = 1 - 1 / R0
  final size z      solves   z = 1 - exp(-R0 * z)
```

```
"""
import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import brentq
import matplotlib.pyplot as plt

beta, gamma = 0.5, 0.2
R0 = beta / gamma
print(f"R0 = {R0},  herd immunity threshold = {1 - 1 / R0:.2f}")


# The final-size equation has no closed form, so find its root numerically
# (R: uniroot).
def final_size(R0):
    if R0 <= 1:
        return 0.0
    return brentq(lambda z: z - (1 - np.exp(-R0 * z)), 1e-6, 1)


print(f"final size from the equation = {final_size(R0):.4f}")


# Check against a simulated epidemic
def sir(t, y, beta, gamma, N):
    S, I, R = y
    return [-beta * S * I / N, beta * S * I / N - gamma * I, gamma * I]


N = 1000
sol = solve_ivp(sir, (0, 300), [N - 1, 1, 0], args=(beta, gamma, N),
                rtol=1e-8, t_eval=np.arange(0, 301))
print(f"final size from the simulation = {sol.y[2, -1] / N:.4f}")

# How the final size depends on R0
print("\n  R0   herd immunity   final size")
for R in [1.2, 1.5, 2, 2.5, 3, 4]:
    print(f"  {R:<4} {1 - 1 / R:>10.3f} {final_size(R):>12.3f}")

R0_grid = np.arange(1.05, 5.01, 0.05)
plt.plot(R0_grid, [final_size(R) for R in R0_grid], label="Final size")
plt.plot(R0_grid, 1 - 1 / R0_grid, "--", label="Herd immunity threshold")
plt.xlabel("R0")
plt.ylabel("Fraction of population")
plt.legend()
plt.show()
# The final size is always larger than the herd immunity threshold:
# the epidemic overshoots because infections continue while I is still large.
```

---



# Vaccination et interventions

Les mesures de contrôle vues comme des modifications du modèle : démarrer avec une partie de la population immunisée, balayer des niveaux de couverture autour du seuil d’immunité collective, et réduire la transmission par la distanciation.

**▼ Voir le R Markdown**

```
```

mathepi/r/04-vaccination.Rmd

````
---
title: "MathEpi 04 - Vaccination and Interventions"
subtitle: "Mathematical Epidemiology Prep (R)"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE, fig.height = 4, fig.width = 7)
```

Control measures change the model's inputs. Vaccination (idealised here as
giving full immunity) moves people straight into the R compartment before the
outbreak, and measures like distancing reduce $\beta$.

```{r}
library(deSolve)

sir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

# Run an epidemic where a fraction `vacc` of the population starts immune
run_sir <- function(beta = 0.5, gamma = 0.2, N = 1000, vacc = 0, days = 200) {
  start <- c(S = N - vacc * N - 1, I = 1, R = vacc * N)
  as.data.frame(ode(start, seq(0, days, by = 1), sir,
                    c(beta = beta, gamma = gamma, N = N)))
}
```

# Compare vaccination coverage

With $R_0 = 2.5$ the herd immunity threshold is $1 - 1/2.5 = 0.6$.

```{r}
coverage <- c(0, 0.3, 0.5, 0.7)
cols <- c("black", "blue", "darkgreen", "red")

plot(NULL, xlim = c(0, 200), ylim = c(0, 350),
     xlab = "Day", ylab = "Infectious people")
for (i in seq_along(coverage)) {
  out <- run_sir(vacc = coverage[i])
  lines(out$time, out$I, col = cols[i], lwd = 2)
}
legend("topright", paste0(coverage * 100, "% vaccinated"), col = cols, lwd = 2)
```

At 70% coverage (above the 60% threshold) the single introduced case cannot
start an epidemic.

# How many people are infected at each coverage level?
````

```{r}
infected_by_coverage <- function(p) {
  out <- run_sir(vacc = p)
  tail(out$R, 1) - p * 1000          # recovered minus those vaccinated
}

p_grid <- seq(0, 0.9, by = 0.05)
total_infected <- sapply(p_grid, infected_by_coverage)

plot(p_grid, total_infected, type = "b", pch = 16,
     xlab = "Vaccination coverage", ylab = "People infected (of 1000)")
abline(v = 0.6, lty = 2)
```

# Reducing transmission instead

A 40% cut in contacts lowers $\beta$ from 0.5 to 0.3, so $R_0$ falls from 2.5
to 1.5:

```{r}
base  <- run_sir(beta = 0.5)
cut40 <- run_sir(beta = 0.3)
c(peak_baseline = round(max(base$I)), peak_with_distancing = round(max(cut40$I)))
```

Distancing flattens and delays the peak even when it does not stop the
epidemic.

**▼ Voir le script Python**

○ ○ ○ mathepi/python/04-vaccination.py

```
"""
MathEpi 04 - Vaccination and interventions

Idealised vaccination: a fraction `vacc` of the population starts fully
immune (in R). Distancing: reduce beta.
With R0 = 2.5 the herd immunity threshold is 1 - 1/2.5 = 0.6.
"""
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt


def sir(t, y, beta, gamma, N):
    S, I, R = y
    return [-beta * S * I / N, beta * S * I / N - gamma * I, gamma * I]


def run_sir(beta=0.5, gamma=0.2, N=1000, vacc=0.0, days=200):
    y0 = [N - vacc * N - 1, 1, vacc * N]
    return solve_ivp(sir, (0, days), y0, args=(beta, gamma, N),
                     rtol=1e-8, t_eval=np.arange(0, days + 1))


# Compare vaccination coverage
for p, colour in zip([0, 0.3, 0.5, 0.7], ["black", "blue", "green", "red"]):
    sol = run_sir(vacc=p)
    plt.plot(sol.t, sol.y[1], color=colour, label=f"{int(p * 100)}% vaccinated")
plt.xlabel("Day")
```

```
plt.ylabel("Infectious people")
plt.legend()
plt.show()


# How many people are infected at each coverage level?
def infected_by_coverage(p, N=1000):
    sol = run_sir(vacc=p, N=N)
    return sol.y[2, -1] - p * N          # recovered minus those vaccinated


p_grid = np.arange(0, 0.95, 0.05)
total_infected = [infected_by_coverage(p) for p in p_grid]

plt.plot(p_grid, total_infected, "o-")
plt.axvline(0.6, linestyle="--", color="grey")
plt.xlabel("Vaccination coverage")
plt.ylabel("People infected (of 1000)")
plt.show()

# Reducing transmission instead: a 40% cut in contacts (beta 0.5 -> 0.3)
base = run_sir(beta=0.5)
cut40 = run_sir(beta=0.3)
print(f"peak, baseline       = {base.y[1].max():.0f}")
print(f"peak, with distancing = {cut40.y[1].max():.0f}")
```

---

Chapitre 05 · 6 sur 8

# Le modèle SEIR

Ajout d’une période de latence avec un compartiment Exposés. Le R0 et la taille finale restent les mêmes, mais l’épidémie est retardée et le pic d’infectieux est plus bas.

**▼ Voir le R Markdown**

○ ○ ○ mathepi/r/05-seir-model.Rmd

````
---
title: "MathEpi 05 - The SEIR Model"
subtitle: "Mathematical Epidemiology Prep (R)"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE, fig.height = 4, fig.width = 7)
```

Many infections have a **latent period**: a person is infected but not yet
infectious. The SEIR model adds an **E**xposed compartment:

$$
\frac{dS}{dt} = -\beta \frac{SI}{N}, \quad
````

````
\frac{dE}{dt} = \beta \frac{SI}{N} - \sigma E, \quad
\frac{dI}{dt} = \sigma E - \gamma I, \quad
\frac{dR}{dt} = \gamma I
$$

where $\sigma = 1/(\text{latent period})$. Here we use a 5-day latent period
($\sigma = 0.2$).

```{r}
library(deSolve)

seir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dE <-  beta * S * I / N - sigma * E
    dI <-  sigma * E - gamma * I
    dR <-  gamma * I
    list(c(dS, dE, dI, dR))
  })
}

sir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
  })
}

N <- 1000
times <- seq(0, 200, by = 1)

seir_out <- as.data.frame(ode(c(S = N - 1, E = 0, I = 1, R = 0), times, seir,
                              c(beta = 0.5, sigma = 0.2, gamma = 0.2, N = N)))
sir_out  <- as.data.frame(ode(c(S = N - 1, I = 1, R = 0), times, sir,
                              c(beta = 0.5, gamma = 0.2, N = N)))
```

# SIR vs SEIR

```{r}
plot(sir_out$time, sir_out$I, type = "l", lwd = 2, col = "red",
     xlab = "Day", ylab = "Infectious people", ylim = c(0, 350))
lines(seir_out$time, seir_out$I, lwd = 2, col = "blue")
legend("topright", c("SIR", "SEIR"), col = c("red", "blue"), lwd = 2)
```

```{r}
data.frame(
  model = c("SIR", "SEIR"),
  peak_day = c(sir_out$time[which.max(sir_out$I)], seir_out$time[which.max(seir_out$I)]),
  peak_infectious = round(c(max(sir_out$I), max(seir_out$I)))
)
```

Both models have the same $R_0 = \beta/\gamma = 2.5$, so they share the same
final size, but the latent period **delays** the epidemic and lowers the peak
of the infectious curve.
````

**▼ Voir le script Python**

○ ○ ○ mathepi/python/05-seir-model.py

```
"""
MathEpi 05 - The SEIR model

Adds an Exposed compartment (infected but not yet infectious):

    dS/dt = -beta * S * I / N
    dE/dt =  beta * S * I / N - sigma * E
    dI/dt =  sigma * E - gamma * I
    dR/dt =  gamma * I

sigma = 1 / (latent period).  Here: 5-day latent period, sigma = 0.2.
"""
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt


def seir(t, y, beta, sigma, gamma, N):
    S, E, I, R = y
    dS = -beta * S * I / N
    dE = beta * S * I / N - sigma * E
    dI = sigma * E - gamma * I
    dR = gamma * I
    return [dS, dE, dI, dR]


def sir(t, y, beta, gamma, N):
    S, I, R = y
    return [-beta * S * I / N, beta * S * I / N - gamma * I, gamma * I]


N = 1000
days = np.arange(0, 201)

seir_sol = solve_ivp(seir, (0, 200), [N - 1, 0, 1, 0],
                     args=(0.5, 0.2, 0.2, N), rtol=1e-8, t_eval=days)
sir_sol = solve_ivp(sir, (0, 200), [N - 1, 1, 0],
                    args=(0.5, 0.2, N), rtol=1e-8, t_eval=days)

plt.plot(days, sir_sol.y[1], "r", label="SIR")
plt.plot(days, seir_sol.y[2], "b", label="SEIR")
plt.xlabel("Day")
plt.ylabel("Infectious people")
plt.legend()
plt.show()

print("model   peak_day  peak_infectious")
print(f"SIR     {days[sir_sol.y[1].argmax()]:>8}  {sir_sol.y[1].max():>10.0f}")
print(f"SEIR    {days[seir_sol.y[2].argmax()]:>8}  {seir_sol.y[2].max():>10.0f}")
# Same R0 = beta / gamma = 2.5, so the same final size,
# but the latent period delays the epidemic and lowers the infectious peak.
```

---



# Épidémies stochastiques (Reed-Frost)

Quand seules quelques personnes sont infectées, le hasard commande. Exécutez un modèle binomial en chaîne des centaines de fois pour voir des flambées

mineures et majeures, et comparez avec la probabilité d’extinction issue de la théorie des processus de branchement.

**▼ Voir le R Markdown**

○ ○ ○ mathepi/r/06-stochastic-reed-frost.Rmd

````
---
title: "MathEpi 06 - Stochastic Epidemics (Reed-Frost)"
subtitle: "Mathematical Epidemiology Prep (R)"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE, fig.height = 4, fig.width = 7)
```

Differential equations describe the average behaviour of a big population.
When only a few people are infected, chance matters: a single case may infect
nobody and the outbreak dies out immediately.

The **Reed-Frost** chain-binomial model captures this in discrete generations.
If $I_t$ people are infectious and each pair of people has probability $p$ of
a transmitting contact, a susceptible escapes infection with probability
$(1-p)^{I_t}$. So

$$ I_{t+1} \sim \text{Binomial}\big(S_t,\; 1 - (1-p)^{I_t}\big). $$

```{r}
reed_frost <- function(N = 200, I0 = 1, p = 0.01) {
  S <- N - I0
  I <- I0
  total <- I0
  while (I > 0) {
    new_I <- rbinom(1, S, 1 - (1 - p)^I)
    S <- S - new_I
    total <- total + new_I
    I <- new_I
  }
  total                             # total number ever infected
}

N <- 200; p <- 0.01
R0 <- (N - 1) * p                   # about 2
R0
```

# Run many outbreaks

```{r}
set.seed(42)
sizes <- replicate(500, reed_frost(N = N, p = p))

hist(sizes, breaks = 30, col = "grey80",
     xlab = "Total infected (out of 200)", main = "500 simulated outbreaks")
```
````

The histogram is **bimodal**: many outbreaks fizzle out early (minor outbreaks), while others take off and infect a large part of the population (major outbreaks).

# Probability of a minor outbreak

```{r}
mean(sizes < 20)                    # fraction of outbreaks that fizzled
```

Early in an outbreak, the number of new cases from each case behaves like a branching process. The probability $q$ that it dies out solves $q = e^{R_0 (q-1)}$:

```{r}
q <- uniroot(function(q) q - exp(R0 * (q - 1)), c(0, 0.999))$root
q                                   # theory
```

Even with $R_0 \approx 2$, roughly one introduction in five goes nowhere. The deterministic SIR model cannot show this, which is why stochastic models are important for small populations and early outbreaks.

**▼ Voir le script Python**

○ ○ ○ mathepi/python/06-stochastic-reed-frost.py

```
"""
MathEpi 06 - Stochastic epidemics (Reed-Frost chain binomial)

With few infected people, chance matters. In discrete generations, if I_t
people are infectious and each pair has probability p of a transmitting
contact, a susceptible escapes with probability (1 - p)**I_t, so

    I_{t+1} ~ Binomial(S_t, 1 - (1 - p)**I_t)
"""
import numpy as np
from scipy.optimize import brentq
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)


def reed_frost(N=200, I0=1, p=0.01):
    S, I, total = N - I0, I0, I0
    while I > 0:
        new_I = rng.binomial(S, 1 - (1 - p) ** I)
        S -= new_I
        total += new_I
        I = new_I
    return total                      # total number ever infected


N, p = 200, 0.01
R0 = (N - 1) * p                      # about 2
print("R0 =", R0)

# Run many outbreaks
sizes = np.array([reed_frost(N=N, p=p) for _ in range(500)])
```

```
plt.hist(sizes, bins=30, color="lightgrey", edgecolor="grey")
plt.xlabel("Total infected (out of 200)")
plt.title("500 simulated outbreaks")
plt.show()
# The histogram is bimodal: minor outbreaks fizzle out, major ones take off.

# Probability of a minor outbreak
print("fraction that fizzled (<20 infected):", np.mean(sizes < 20))

# Early on the epidemic behaves like a branching process; the probability q
# that it dies out solves   q = exp(R0 * (q - 1)).
q = brentq(lambda q: q - np.exp(R0 * (q - 1)), 0, 0.999)
print("theory:", round(q, 3))
# Even with R0 ~ 2, roughly one introduction in five goes nowhere.
```

---

Chapitre 07 · 8 sur 8

# Ajuster un modèle à des données

Estimer un taux de transmission à partir de données par les moindres carrés : simuler des observations bruitées d'une épidémie connue, minimiser l'erreur quadratique, et retrouver β et R0.

**▼ Voir le R Markdown**

○ ○ ○ mathepi/r/07-fitting-sir.Rmd

````
---
title: "MathEpi 07 - Fitting a Model to Data"
subtitle: "Mathematical Epidemiology Prep (R)"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
    highlight: tango
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE, fig.height = 4, fig.width = 7)
```

Models become useful when their parameters are estimated from data. Here we
generate "observed" data from an SIR epidemic with known parameters, then
recover the transmission rate $\beta$ by **least squares**: choose the
$\beta$ that makes the model curve closest to the observations.

# Simulate noisy observations

```{r}
library(deSolve)

sir <- function(time, state, parms) {
  with(as.list(c(state, parms)), {
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
````
````

````
  })
}

N <- 1000
gamma <- 0.2                 # assume the infectious period is known (5 days)
days <- 0:60

model_I <- function(beta) {
  out <- ode(c(S = N - 1, I = 1, R = 0), days, sir,
             c(beta = beta, gamma = gamma, N = N))
  out[, "I"]
}

set.seed(10)
true_beta <- 0.5
observed <- rpois(length(days), model_I(true_beta))   # Poisson observation noise
```

# Define and minimise the error

```{r}
sse <- function(beta) sum((observed - model_I(beta))^2)

fit <- optimize(sse, interval = c(0.1, 1))
beta_hat <- fit$minimum
beta_hat                          # close to the true value 0.5
beta_hat / gamma                  # estimated R0 (true value 2.5)
```

# Compare the fitted curve with the data

```{r}
plot(days, observed, pch = 16, xlab = "Day", ylab = "Infectious people")
lines(days, model_I(beta_hat), col = "red", lwd = 2)
legend("topright", c("Observed", "Fitted SIR"), pch = c(16, NA),
       lty = c(NA, 1), col = c("black", "red"))
```

# See why it works: the error curve

```{r}
beta_grid <- seq(0.3, 0.7, by = 0.01)
plot(beta_grid, sapply(beta_grid, sse), type = "l", lwd = 2,
     xlab = "beta", ylab = "Sum of squared errors")
abline(v = beta_hat, lty = 2, col = "red")
```

The error has a clear minimum near the true $\beta$. In a real analysis you
would also estimate $\gamma$, report uncertainty, and use a likelihood
appropriate for count data, but the logic is exactly this.
````

**▼ Voir le script Python**

○ ○ ○ mathepi/python/07-fitting-sir.py

```
"""
MathEpi 07 - Fitting a model to data (least squares)

Generate "observed" data from an SIR epidemic with known parameters, then
recover the transmission rate beta by choosing the value that makes the
model curve closest to the data.
```
```

```
"""
import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import minimize_scalar
import matplotlib.pyplot as plt


def sir(t, y, beta, gamma, N):
    S, I, R = y
    return [-beta * S * I / N, beta * S * I / N - gamma * I, gamma * I]


N = 1000
gamma = 0.2                     # assume the infectious period is known (5 days)
days = np.arange(0, 61)


def model_I(beta):
    sol = solve_ivp(sir, (0, 60), [N - 1, 1, 0], args=(beta, gamma, N), rtol=1e-8, t_eval=days)
    return sol.y[1]


# Simulate noisy observations (Poisson noise around the true curve)
rng = np.random.default_rng(10)
true_beta = 0.5
observed = rng.poisson(model_I(true_beta))


# Least squares: pick the beta that minimises the sum of squared errors
def sse(beta):
    return np.sum((observed - model_I(beta)) ** 2)


fit = minimize_scalar(sse, bounds=(0.1, 1.0), method="bounded")
beta_hat = fit.x
print(f"estimated beta = {beta_hat:.4f}   (true {true_beta})")
print(f"estimated R0   = {beta_hat / gamma:.3f}   (true 2.5)")

plt.plot(days, observed, "ko", label="Observed")
plt.plot(days, model_I(beta_hat), "r-", label="Fitted SIR")
plt.xlabel("Day")
plt.ylabel("Infectious people")
plt.legend()
plt.show()

# The error curve has a clear minimum near the true beta
beta_grid = np.arange(0.3, 0.701, 0.01)
plt.plot(beta_grid, [sse(b) for b in beta_grid])
plt.axvline(beta_hat, linestyle="--", color="red")
plt.xlabel("beta")
plt.ylabel("Sum of squared errors")
plt.show()
```

# Dr. Lema Logamou Seknewna

AIMS-NISR Data Scientist, Researcher and Lecturer

African Institute for Mathematical Sciences, Research and Innovation Centre